



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Никола Јоловић

**Колаборативна *Juruter* биљежница са
подршком за уређивање и извршавање
у реалном времену**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Никола Јоловић	Број индекса:	SV9/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Колаборативна <i>Jupyter</i> биљежница са подршком за уређивање и извршавање у реалном времену
--

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Никола Јоловић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Колаборативна <i>Jupyter</i> биљежница са подршком за уређивање и извршавање у реалном времену
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	7/61/45/2/18/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Јирутер биљежница, сарадња у реалном времену, <i>CRDT</i> , Операционе трансформације
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај рад представља студију случаја софтвера <i>Crab Collab</i> , колаборативне <i>Jupyter</i> биљежнице. <i>Crab Collab</i> омогућава да више корисника уређују један документ у реалном времену, уз одржавање конзистентног погледа за све кориснике. Архитектура са централним сервером омогућава дијелење окружења за извршавање кода и пружа гаранције потребне за синхронизацију стања. Представљено рјешење користи структуру документа типа биљежнице да комбинује приступе за разрјешавање конфликта засноване на операционим трансформацијама и фракционом индексирању ради постизања конзистентности.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Nikola Jolović
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	A Collaborative Jupyter Notebook with real-time editing and execution support
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/61/45/2/18/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Jupyter notebook, real-time collaboration, CRDT, Operational transformation
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis presents a case study of <i>Crab Collab</i> , a collaborative <i>Jupyter</i> notebook. <i>Crab Collab</i> allows multiple users to edit one document in real-time, while maintaining a consistent view for all users. Architecture with a central server allows sharing a code execution environment and provides guarantees required for state synchronization. The presented solution utilizes notebook document structure to combine conflict resolution strategies based on operational transformations and fractional indexing to achieve consistency.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	1
1.2	Циљ рада	1
1.3	Опсег рада	1
1.4	Структура рада	1
2	Теоријске основе	3
2.1	Софтвер за групни рад у реалном времену	3
2.2	<i>Jupyter</i>	6
2.3	Технологије	8
3	Постојећа рјешења	11
3.1	<i>Google Colab</i>	11
3.2	<i>JupyterLab</i> у реалном времену	11
3.3	<i>CoCalc</i>	12
3.4	<i>Google Docs</i>	12
3.5	<i>Figma</i>	13
3.6	Закључак поглавља	13
4	Функционални и нефункционални захтјеви	15
4.1	Функционални захтјеви	15
4.2	Нефункционални захтјеви	17
4.3	Ставке ван опсега	17
5	Архитектура система	19
5.1	Преглед архитектуре	19
5.2	Модел стања	19
5.3	Модел комуникације	20
5.4	Структура компоненти	22
5.5	Преглед усвојених механизма	23
6	Имплементација	25
6.1	Организација изворног кода	25
6.2	Апстракција стања биљежнице	26
6.3	Поредак ћелија	28
6.4	Текстуални садржај ћелија	31
6.5	Позиције курсора	35
6.6	Извршавање кода	36
6.7	Присуство учесника	41
7	Закључак	43
	Списак слика	45
	Списак листинга	47
	Списак табела	49
	Списак коришћених скраћеница	51
	Списак коришћених појмова	53
	Биографија	55
	Литература	57
	TODOs	61

1.1 Мотивација

Рачунарска биљежница (енг. *computational notebook*) је тип дигиталног документа који омогућава комбинавање форматираног текста, изворног кода и резултата извршавања у једном интерактивном документу [1]. Овај концепт остварују алати попут *Jupyter*-а, *R Markdown*-а и *Mathematica* биљежница. У остатку рада, појам рачунарска биљежница подразумева компатибилност са *Jupyter* стандардима.

Jupyter је пројекат који дефинише стандарде и развија алате за рад са рачунарским биљежницама [2]. *Jupyter* биљежница је дизајнирана да олакша документовање, дијелење и репродуковање анализе података [3]. То је чини широко распрострањеном у области науке о подацима [4], као и у образовању [5].

Сарадња у реалном времену подразумева да више корисника на потенцијално удаљеним локацијама уређује исти дигитални садржај и одмах види измјене које уносе остали корисници. Примјери софтвера који омогућавају овај начин рада су *Google Docs* [6] за уређивање текстуалних докумената и *Figma* [7] за уређивање графичких докумената. Распрострањеност ових и њима сличних алата говори о реалној потреби за оваквим приступом при уређивању свих врста дигиталних докумената.

Сарадња у реалном времену на *Jupyter* биљежницама олакшава дијелење контекста, подстиче истраживање и смањује трошкове комуникације. Упркос томе, постоје изазови који могу довести до ометања међу корисницима, што отежава уклањање грешака и омета ток рада [8]. Структура документа у коме су текст, код и стање извршавања тијесно повезани чини ове проблеме израженијим него код једноставнијих текстуалних докумената.

1.2 Циљ рада

Овај рад представља студију случаја софтвера *Crab Collab*, алата за колаборативно уређивање *Jupyter* биљежница у реалном времену. Циљ рада је да се, кроз развој система истраже механизми за рјешавање конфликта приликом истовремених измјена у циљу одржавања конзистентности погледа на документ, као и интеграција са дијелом система за извршавање кода.

1.3 Опсег рада

Систем представљен у овом раду реализован је као доказ концепта (енг. *proof-of-concept*, *PoC*), а не као систем спреман за продукцијску употребу. Разлог за то је очување фокуса на рјешавању кључних проблема подршке колаборације над рачунарском биљежницом у реалном времену, без увођења додатне комплексности која би сакрила суштину рјешења.

Ова одлука има директне импликације на архитектонске и имплементационе одлуке изнесене у наставку текста. У погледу постојаности података, скалабилности, управљања сесијама и комплетности скупа функционалности, усвојена су упрошћена рјешења довољна за демонстрацију концепта али не и стварну продукцијску примјену. Ставке које су услед тога изостављене из опсега, уз разлоге изостављања, наведене су у одјељку 4.3, док је осврт на ограничења која из тога произлазе дат у поглављу 7.

1.4 Структура рада

Рад је организован у седам поглавља која воде читаоца од теоријских основа потребних за разумијевање рјешења до конкретне имплементације система, закључујући оцјеном постигнутог рјешења и правцима даљег развоја.

Након увода, друго поглавље нуди објашњење теоријских концепата потребних за разумијевање остатка рада, укључујући основе система за колаборацију у реалном времену, као и детаљнији увид у *Jupyter* модел и концепте. Дат је и преглед технологија коришћених за израду рјешења.

Треће поглавље анализира постојећа рјешења. Идентификоване су разлике између њих и онога што *Crab Collab* жели да постигне, али и преузете кључне идеје о архитектури и механизмима синхронизације.

Четврто поглавље пружа преглед функционалних и нефункционалних захтјева система. Наведене су и ставке које су експлицитно ван опсега.

Пето поглавље представља архитектуру система. Описане су основне компоненте и њихови међусобни односи и дат је преглед коришћених архитектуралних образаца.

Шесто поглавље се бави имплементацијом система. Објашњене су стратегије за рјешавање конфликта уз детаље имплементираних алгоритама, као и детаљи интеграције окружења за извршавање кода.

Рад се завршава у седмом поглављу, у коме су процијењене предности и мане представљеног рјешења и дате финалне смјернице за будући развој.

Ово поглавље описује теоријске основе неопходне за разумијевање рјешења и даје преглед коришћених технологија.

2.1 Софтвер за групни рад у реалном времену

Прије него што буду детаљно представљене конкретне технике за рјешавање конфликта, потребно је увести терминологију на којој су засноване. Овај одјељак дефинише појмове који се односе на групни рад у реалном времену, затим објашњава зашто синхрона сарадња захтијева аутоматско рјешавање конфликта и на крају нуди опис два доминантна алгоритамска приступа том проблему.

2.1.1 Групвер и групвер у реалном времену

Групвер (енг. *groupware*) је софтвер намијењен подршци рада више корисника на заједничком задатку, дијелећи приступ заједничком окружењу. Карактеристика која одваја групвер од других вишекорисничких система је обавјештавање о акцијама других корисника. Примјери вишекорисничких система који не спадају у групвер су системи за управљање базама података и оперативни системи са расподелом времена јер пружају ограничену подршку за обавјештења о акцијама. Обично се захтијева додатни упит ка систему да би се дошло до информације о акцији другог корисника [9].

Појединачна употреба групвер система назива се сесија. Сесију у сваком тренутку чини скуп учесника којима систем обезбјеђује интерфејс ка дијеленом садржају. На примјер, корисници система могу видјети синхронизован приказ података који се мијењају кроз вријеме [9].

Вријеме потребно систему да се акције корисника одразе у његовом сопственом интерфејсу назива се вријеме одзива (енг. *response time*). Вријеме потребно да промјене од стране једног корисника доспију у интерфејс других корисника назива се вријеме обавјештења (енг. *notification time*) [9].

Групвер у реалном времену је ужа категорија која подразумева да се обавјештења о корисничким акцијама брзо пропaгирају другим корисницима. Карактеришу га сљедеће особине:

- интерактивност: вријеме одзива мора бити кратко;
- рад у реалном времену: вријеме обавјештења мора бити упоредиво са временом одзива;
- дистрибуираност: учесници могу бити географски удаљени и немају приступ истој машини, него комуникација мора да се одвија преко рачунарске мреже;
- фокусираност: корисници често приступају истим подацима у исто вријеме [9].

2.1.2 Контрола конкурентности

Контрола конкурентности означава скуп техника којима се обезбјеђује исправно извршавање операција над дијеленим стањем и поред тога што се оне извршавају конкурентно, тј. без унапријед задатог редослиједа. Механизам контроле конкурентности пружа корисницима поглед на стање такав да су њихове операције извршаване једна за другом, иако се у стварности могу преклапати у времену [10].

Конфликт је стање у које се долази када двије или више конкурентних операција дјелују на исти дио дијеленог стања на начин који зависи од редослиједа њиховог извршавања, тј. када би различит редослијед примјене довео до различитог или неисправног резултата.

Механизме за контролу конкурентности можемо подијелити у двије категорије:

- песимистички, тј. засновани на превенцији конфликта и
- оптимистички, тј. засновани на разрјешавању конфликта [11].

Типичан песимистички механизам јесте закључавање (енг. *locking*). Закључавањем критичног ресурса, само један учесник може да дјелује на њега, тако да не може да настане разлика у стању за различите учеснике.

Оптимистички механизми подразумевају да се операције извршавају без рестрикција, те да се конфликти у стању детектују и ретроактивно отклоне. Овај приступ се ослања на претпоставку да до конфликта неће доћи у већини случајева, па ће потреба за примјеном самог механизма бити мања него уколико наметнемо обавезну контролу као код песимистичких приступа [11].

2.1.3 Захтјеви за сарадњу у реалном времену

Групвер системи се у зависности од временске димензије сарадње могу подијелити на синхроне и асинхроне [12]. Синхрони системи захтијевају одговор интерфејса одмах, док асинхрони системи, попут система за верзионисање, вики платформи или електронске поште, то не захтијевају, чиме је омогућено одлагање усклађивања конкурентних измјена истих подака, а често и ручни преглед и спајање измјена.

Захтјев за интерактивношћу и радом у реалном времену чини групвер системе синхроним [12]. У комбинацији са фокусираношћу сесија, намећу се специфични захтјеви за контролу конкурентности.

Интерактивност захтијева вријеме одзива довољно мало да корисницима дјелује тренутно, што имплицира употребу оптимистичких механизма контроле конкурентности.

У таквом окружењу фокусираност неизбијежно доводи до конфликта у стању, чије се елиминисање захтијева у реалном времену. Стога, конфликти морају бити ријешени аутоматски и без уочљивог кашњења [9].

За постизање ових захтјева у литератури се издвајају два доминантна алгоритамска приступа, описана у наредним одјељцима: операциона трансформација и *CRDT* (енг. *Conflict-free Replicated Data Types*) структуре података.

2.1.4 Операциона трансформација

Операциона трансформација (енг. *Operational transformation*, *OT*) представља оптимистички механизам за контролу конкурентности у групвер системима [9]. Основна идеја јесте да се примљена измјена трансформише у односу на конкурентне измјене примљене прије ње, тако да њен утицај остане исправан, иако је стање на које се примјењује промијењено у односу на оно над којим је измјена настала.

Разлог зашто је трансформација неопходна илустрован је следећим примјером. Нека дијељени документ представља 0-индексиран низ карактера и нека је његово почетно стање "абв". Нека корисник А уметне карактер "г" на позицију 0. Тада стање документа треба да буде "габв". Истовремено, корисник Б обрише карактер на позицији 1. Његова намјера је да обрише карактер "б". Међутим, уколико измјена корисника А буде примјењена прва, измјена корисника Б ће обрисати карактер "а" и резултујуће стање ће бити "Гбв", умјесто жељеног стања "Гав". Проблем се може ријешити трансформацијом операције корисника Б тако да брише карактер на позицији 2. За детаљнији преглед трансформације сложенијих комбинација измјена, погледати [13].

Да би систем заснован на *OT*-у био исправан, потребно је задовољити двије особине: очување узрочности и конвергенцију. Очување узрочности осигурава да се узрочно повезане измјене извршавају истим редоследом на свим мјестима. Конвергенција осигурава да ће, након што сви корисници приме исти скуп измјена, њихове копије документа бити идентичне [14].

Конвергенција конкурентних измјена зависи од исправности функције трансформације. Њена исправност формализована је кроз двије особине *TP1* и *TP2*. *TP1* захтијева да за двије конкурентне измјене важи да примјена резултата трансформације прве измјене у односу на другу резултује стањем еквивалентним оном које се добија примјеном резултата трансформације друге измјене у односу на прву. *TP2* захтијева

да трансформација треће конкурентне измјене у односу на низ од претходне двије даје идентичан резултат без обзира на редослијед операција у том низу.

TP2 је, међутим, неопходна искључиво у децентрализованим системима, гдје измјене могу стићи на различита мјеста различитим редослиједом. Уколико се систем ослања на централни сервер који успоставља јединствен, тотални редослијед свих измјена, довољно је задовољити само *TP1* [15]. Овај образац представља основу протокола коришћеног у *Google Docs*-у, гдје је управо ослањање на централни сервер истакнуто као разлог зашто сложеност обраде измјена не расте са бројем истовремених корисника [16].

Испуњавање *TP1* и *TP2* историјски се показало тежим него што се чини. Томе свједочи чињеница да сама изворна функција трансформације коју су предложили Елис и Гибс не задовољава ни *TP1* ни *TP2* [14]. Комплексност дизајна функција трансформације, чак и у централизованим поставкама, доноси потешкоће и захтијева пажљиво пројектовање и ригорозне доказе како би се гарантовала њихова исправност. Ово представља кључно ограничење ОТ приступа, које *CRDT* структуре, описане у наредном одјељку, рјешавају на другачији начин.

2.1.5 *CRDT* и сродни приступи

2.1.5.1 *CRDT*

CRDT (енг. *Conflict-free Replicated Data Type*) представља структуру података која у дистрибуираном систему гарантује конвергенцију свих реплика структуре стриктно самим њеним математичким својствима, а не засебно доказаним алгоритмом. Систем заснован на *CRDT* мора задовољити три захтјева:

- Свака реплика структуре података мора бити у могућности да се ажурира конкурентно и независно од осталих, без било какве координације.
- Неконзистентности стања мора аутоматски да ријеша механизам који је дио саме структуре података.
- Без обзира што у датом тренутку могу имати различито стање, све реплике морају конвергирати ка истом резултату [17].

Једноставан примјер *CRDT* структуре је скуп који подржава само додавање елемената. Пошто је додавање истог елемента идемпотентно, а унија скупова комутативна и асоцијативна, коначно стање скупа не зависи од редослиједа нити броја понављања примљених ажурирања. Свака реплика конвергира ка истом скупу простом унијом свих додатих елемената.

Гаранција коју *CRDT* структуре пружају назива се снажна коначна конзистентност (енг. *strong eventual consistency*). Реплике које су примиле исти скуп операција биће гарантовано у истом стању, без потребе за накнадним усклађивањем. Ова гаранција, међутим, обично захтијева додатне метаподатке и механизме за рјешавање конфликта код конкурентних измјена, што их чини захтјевним са становишта коришћене меморије и комплексности алгоритама. Примјери додатних података и механизма су јединствени идентификатори елемената, ознаке за обрисане елементе (енг. *tombstones*), векторски сат (енг. *vector clock*) и сл.

2.1.5.2 *CRDT* у контексту централизованих система

Постојање централног сервера пружа додатне гаранције које често омогућавају измјене алгоритама уклањањем метаподатака и комплексних механизма. На примјер, ако сервер уведе тотални поредак операција, елиминише се потреба за векторским сатовима. То омогућава поједностављену имплементацију и смањену употребу рачунарских ресурса [7].

Овакве структуре се не могу стриктно назвати *CRDT*, али су директно инспирисане њима и релевантна литература представља вриједан извор у том контексту. Она нуди провјерен, математички поткован темељ на ком се граде поједностављене структуре прилагођене потребама конкретних система [7].

2.1.5.3 Фракционо индексирање

Фракционо индексирање (енг. *fractional indexing*) је техника за одржавање редослиједа у листи, без потребе да се велики број елемената ренумерише при одређеним операцијама. Позиције елемената листе представљене су реалним бројевима, тако да је увијек могуће уметнути елемент између друга два [18].

Када се овој техници придружи детерминистичка техника разрјешавања конкурентних додјела исте позиције, добија се структура података која гарантује конвергенцију. Ово се може постићи додавањем метаподатака, чиме добијамо *CRDT*, или увођењем тоталног поретка помоћу централног сервера, чиме добијамо прагматичну структуру инспирисану *CRDT* принципима.

2.1.6 Поређење ОТ и *CRDT* приступа

Операциона трансформација и *CRDT* су алати који нам омогућују аутоматско рјешавање конфликта код софтвера за сарадњу у реалном времену. Два различита приступа нуде различите гаранције и имају различиту цијену употребе.

Код ОТ-а, конвергенција зависи од исправности функције трансформације, те је у њу потребно уложити посебан инжењерски напор за пројектовање и доказ коректности, што се историјски показало склоно грешкама. Код *CRDT*-а, конвергенција слиједи директно из математичких својстава структуре података, без потребе за засебним доказом исправности [19].

С друге стране, *CRDT* структуре уносе додатно оптерећење метаподацима неопходно за рјешавање конфликта без централног ауторитета. У поређењу с њима, ОТ операције обично користе мање рачунарских ресурса, али су уско семантички везане за конкретан тип података над којим дјелују.

Ове разлике се, међутим, смањују у централизованог архитектури. Постојање централног ауторитета поједностављује захтјеве наметнуте изворно пројектованим, децентрализованим верзијама оба приступа.

2.2 Jupyter

Овај одјељак представља *Jupyter* пројекат и његове компоненте, са посебним освртом на структуру докумената и архитектуру неопходну за разумијевање интеграције извршавања кода описане у поглављу 6.

2.2.1 Jupyter пројекат

Jupyter пројекат обухвата четири компоненте:

- формат документа,
- кернел,
- скуп корисничких апликација и
- протокол који их повезује [2].

Формат документа представља начин записа садржаја независан од конкретне апликације. Назива се биљежница [2].

Кернел (енг. *kernel*) је засебан процес који извршава код и враћа резултате. Кернел одржава стање у меморији између више извршавања кода. Максимално једна ћелија се може извршавати у датом тренутку [2].

Корисничка апликација, често названа и *Jupyter* фронтенд (енг. *front-end*), представља кориснички интерфејс за рад са биљежницама. Примјери су *Jupyter Notebook* и *JupyterLab* [2].

Кернел и корисничка апликација комуницирају протоколом који је описан у наставку [2].

2.2.2 Биљежница (документ)

Документ типа биљежнице организован је као низ ћелија. Ћелије могу бити један од два типа, зависно од њиховог садржаја:

- ћелије кода, које садрже изворни код и
- текстуалне ћелије, које садрже форматиран текст у маркдаун (енг. *markdown*) формату [2].

Ћелије кода могуће је извршити, што подразумева извршавање кода који је њихов садржај. Како је сваку појединачну ћелију могуће извршити, редослијед извршавања не зависи од редослиједа ћелија у документу. Свака ћелија чува и редни број извршавања (енг. *execution count*) који биљежи редослијед којим су ћелије извршене [2].

Извршавање ћелије са кодом производи излаз (енг. *output*) који се придружује тој ћелији. Излаз није ограничен на текстуални садржај. Може укључивати и слике, графике или *HTML* садржај [2].

2.2.3 ZeroMQ

ZeroMQ је библиотека за размјену порука која, за разлику од традиционалних система реда порука, не захтијева посредни процес (енг. *broker*) за преношење порука [20]. Редови порука држе се у самој меморији програма.

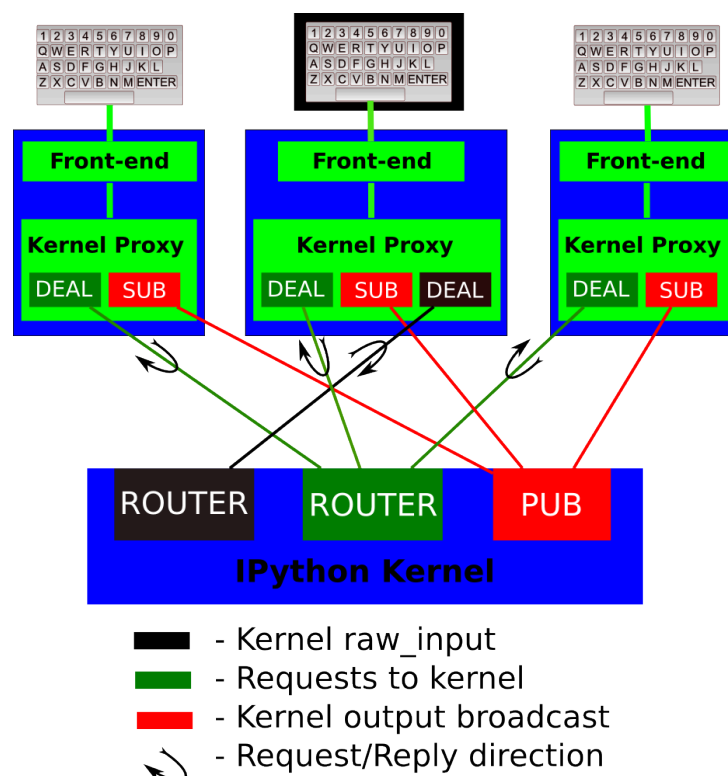
Сокет (енг. *socket*) у *ZeroMQ* је примитива за крајњу тачку комуникације. Представља асинхрону апстракцију реда порука [20]. Постоји више врста сокета и сваки подразумева засебан образац у комуникацији. Овдје је дат преглед типова сокета коришћених у *Jupyter* протоколу.

Тип захтјев-одговор (енг. *request-reply*) функционише по строгој наизмјеничној комуникацији. Једна страна шаље захтјев и очекује одговор прије него што може да пошаље сљедећи захтјев.

Тип објава-претплата (енг. *publish-subscribe*) користи истоимени образац да омогући да поруке које емитује једна страна буду примљене од стране произвољног броја претплаћених страна.

Строга наизмјеничност основног обрасца захтјев-одговор ограничава комуникацију на један неријешен захтјев у сваком тренутку. *ZeroMQ* пружа асинхронно проширење овог обрасца путем сокета типа *DEALER* и *ROUTER*, који омогућавају слање произвољног броја захтјева без чекања на одговоре.

2.2.4 Jupyter протокол



Слика 1: Архитектура комуникације између *Jupyter* фронтенда и кернела [21]¹

¹Извор: *Jupyter Development Team*. Слика је доступна под условима *Modified BSD* лиценце. Доступно на: <https://jupyter-client.readthedocs.io/en/latest/>. Приступљено: 24. августа 2026. год.

Jupyter протокол дефинише начин и формат комуникације између корисничке апликације (*Jupyter* фронтенда) и кернела [21]. Протокол је заснован на асинхроној размјени порука преко *ZeroMQ* сокета, што омогућава да се један кернел истовремено повеже са више корисничких апликација. Општа архитектура ове комуникације и распоред канала приказани су на слици 1.

Да би се спријечило блокирање комуникације у случају дуготрајних извршавања и раздвојили различити типови саобраћаја, *Jupyter* протокол користи пет засебних канала, односно *ZeroMQ* сокета [21]:

- **Shell:** *ROUTER* сокет који се користи за примарну комуникацију по обрасцу захтјев-одговор. Преко овог канала фронтенд шаље захтјеве за извршавање кода, инспекцију објеката и сл, док кернел враћа одговарајуће одговоре.
- **IOPub:** *XPUB/PUB* сокет који служи као канал за емитовање свим повезаним фронтендовима. Кернел преко овог канала објављује бочне ефекте извршавања кода (стандардни излаз, грешке, графичке приказе), као и промјене сопственог стања (нпр. да ли је заузет или слободан). На овај начин сви корисници у сесији могу видјети резултате извршавања у реалном времену.
- **Stdin:** *ROUTER* сокет који омогућава кернелу да затражи унос од корисника (нпр. при позиву Пајтон функције `input()`). Захтјев се шаље искључиво фронтенду који је покренуо извршавање, а који затим шаље кориснички унос назад кернелу преко *DEALER* сокета.
- **Control:** *ROUTER* сокет који функционише аналогно *Shell* каналу, али ради на засебној нити. Ово омогућава да се поруке високог приоритета, попут прекида извршавања (енг. *interrupt*) или гашења кернела (енг. *shutdown*), обраде тренутно без чекања да се заврши извршавање претходно започетих захтјева на *Shell* каналу.
- **Heartbeat:** Захтјев-одговор сокет који служи за контролу присутности (енг. *heartbeat*). Корисничка апликација и кернел размјењују једноставне поруке како би се тренутно детектовао евентуални губитак мрежне везе или пад кернел процеса.

Поруке које се размјењују преко ових канала логички су форматиране као *JSON* објекти и садрже сљедеће компоненте [21]:

- **header:** садржи метаподатке о самој поруци, укључујући јединствени идентификатор поруке (`msg_id`), идентификатор сесије (`session`), тип поруке (`msg_type`), верзију протокола и временску ознаку.
- **parent_header:** за поруке које су директан одговор или споредни ефекат другог захтјева, ово поље садржи заглавље изворне поруке. Ово омогућава фронтенду да тачно повеже генерисани излаз са одговарајућом ћелијом у документу.
- **metadata:** опциони *JSON* објекат са додатним контекстуалним информацијама о поруци.
- **content:** главни садржај поруке чија структура зависи од дефинисаног типа поруке (`msg_type`).
- **buffers:** опциони низ сирових бинарних бафера за ефикасан пренос већих количина података без *JSON* енкодирања.

На нивоу мрежног преноса (енг. *wire protocol*), порука се серијализује у низ *ZeroMQ* бајт-фрејмова раздвојених делимитером `<IDS|MSG>`. Ради безбједности, свака порука садржи и *HMAC* дигитални потпис (најчешће заснован на *SHA-256* хеш функцији) који се израчунава помоћу тајног кључа дефинисаног у датотеци повезивања (енг. *connection file*). Тиме се осигурава да кернел извршава само наредбе које потичу од овлашћених корисника [21].

2.3 Технологије

Овај одјељак даје преглед технологија коришћених у имплементацији. Обухваћене су само оне чија својства носе рјешења описана у поглављу 6.

2.3.1 Раст

Раст (енг. *Rust*) је системски програмски језик опште намјене. Управљање меморијом провјерава се у фази превожјења, системом власништва (енг. *ownership*) и позајмљивања (енг. *borrowing*). Свака вриједност има тачно једног власника, а приступ преко референци ограничен је тако да у датом тренутку постоји или

произвољан број референци за читање или тачно једна за упис. Тиме се, поред безбједности меморије, у фази превођења спречавају и трке за подацима (енг. data race) у вишенитним програмима [22].

Особина (енг. trait) је именовани скуп функција које тип мора имплементирати. Служи за апстракцију понашања независно од конкретног типа [23].

Cargo је алат за превођење и управљање зависностима. Јединица превођења и дистрибуције назива се сандук (енг. crate), а више сандука који се преводе заједно чине радни простор (енг. workspace). Обиљежје (енг. feature) је именовани скуп функционалности сандука који се укључује или искључује при превођењу, чиме се из истог изворног кода добијају различите варијанте сандука [23].

2.3.2 Асинхронно извршавање и Tokio

Асинхронно извршавање омогућава да програм, умјесто да блокира нит чекајући завршетак улазно-излазне операције, ту нит уступи другом послу. Функције које могу да чекају означавају се у Расту кључном ријечју `async`, а мјеста чекања изразом `await`.

Јединица посла назива се задатак (енг. task). За разлику од нити, задатку није потребан засебан стек оперативног система, па је његово покретање знатно јефтиније, а број истовремених задатака може бити много већи од броја нити. Задатке на расположиве нити распоређује извршилац (енг. runtime).

Tokio је асинхрони извршилац за Раст. Поред распоређивања задатака, пружа и канале (енг. channel), редове порука којима задаци међусобно комуницирају без дијелења промјенљивог стања [24].

2.3.3 WebSocket

WebSocket је протокол који над једном TCP везом успоставља двосмјерну комуникацију (енг. full-duplex) између веб-прегледача и сервера. Веза се успоставља надоградњом (енг. upgrade) почетног HTTP захтјева, након чега обје стране шаљу поруке у произвољном тренутку, без претходног захтјева друге стране. Комуникација је оријентисана на поруке, а не на бајт-ток: протокол сам одржава границе порука [25].

Пошто се одвија над једном TCP везом, пренос је поуздан и поруке стижу редослиједом којим су послате, док год веза траје. Ова гаранција важи искључиво унутар једне везе. Протокол не уређује међусобни редослијед порука различитих клијената, нити даје било какву гаранцију о порукама послатим прије прекида и поновног успостављања везе.

2.3.4 Тајпскрипт и React

Тајпскрипт (енг. TypeScript) је надскуп Јаваскрипта који уводи статичке типове. Типови се провјеравају при превођењу и уклањају из резултата, који је обичан Јаваскрипт [26].

React је библиотека за израду корисничких интерфејса. Интерфејс се описује декларативно, стаблом компоненти. Компонента је функција која на основу стања враћаја опис приказа. Промјена стања покреће поновно израчунавање само оних компоненти које то стање читају. Кука (енг. hook) је функција која компоненти даје приступ стању и споредним ефектима изван самог исцртавања [27].

Monaco је готова компонента уређивача текста, изворно развијена за Visual Studio Code, која пружа приказ изворног кода са означавањем синтаксе [28].

2.3.5 Zustand и Immer

Zustand је библиотека која стање држи у складишту изван стабла компоненти. Компонента се претплаћује на дио стања и поново се исцртава само када се тај дио промијени [29].

Immer омогућава да се непромјенљива вриједност ажурира тако што се измјена опише над привременим нацртом (енг. draft), као да се подаци мијењају на лицу мјеста. Резултат је нова вриједност, при чему непромијењени дијелови остају дијелени са претходном [30]. Тако се провјера да ли је дио стања промијењен своди на поређење референци.

2.3.6 WebAssembly

WebAssembly је преносив бинарни формат инструкција, пројектован као циљ превођења за системске језике попут Раста. Извршава се у изолованом окружењу (енг. *sandbox*) веб-прегледача, брзином блиском изворном извршавању [\[31\]](#).

Модул у WebAssembly формату комуницира са Јаваскриптом преко експлицитно извезених функција и дијелене линеарне меморије. За Раст, генерисање тог омотача аутоматизује библиотека `wasm_bindgen`. Тиме је исти изворни код могуће превести двапут: изворно, за сервер, и у модул који се извршава у прегледачу.

У овом поглављу су описана постојећа рјешења која верификовано функционишу у продукционом окружењу. Укључени су примјери дијељених *Jupyter* биљежница, као и примјери из других домена који су послужили као инспирација за архитектуру и механизме коришћене у *Crab Collab*.

3.1 Google Colab

Најпознатији софтвер за онлајн дијељене *Jupyter* биљежнице је *Google Colab*. Пружа веб-интерфејс за рад са биљежницама и нуди приступ једном документу за више корисника.

Упркос распрострањености употребе и самом називу софтвера, *Google Colab* има ограничену подршку за колаборацију у реалном времену. Конкурентне измјене садржаја исте ћелије од стране више корисника разрјешавају се по принципу „последњи запис побеђује” (енг. *last-write-wins*, *LWW*), који подразумева да последња измјена одређује коначно стање и „препише” преко свега што је примљено раније али га последња измјена није била свјесна.

Постоји још софтвера за дијељене биљежнице са ограниченом подршком за сарадњу у реалном времену. *Deepnote* и *Hex* рјешавају проблем конкурентних измјена песимистичким закључавањем на нивоу ћелије, што значи да никада не може више корисника да прави измјене у садржају исте ћелије [32].

3.2 JupyterLab у реалном времену

JupyterLab је апликација која пружа веб-интерфејс за рад са *Jupyter* биљежницама. Архитектура софтвера укључује веб-интерфејс у прегледачу (клијент), серверску компоненту на коју се клијенти прикључују и *Jupyter* кернел којим управља сервер. Сервер има улогу медијатора између клијента, са којим комуницира преко *WebSocket* везе, и кернела са којим комуницира преко *Jupyter* протокола.

Подршка за сарадњу у реалном времену реализована је преко проширења *jupyter-collaboration*, које има серверску и клијентску компоненту. Додаје се нова, засебна *WebSocket* веза између сервера и клијента преко које се врши комуникација потребна на синхронизацију стања. Више клијената може да се повеже на један сервер, што није предвиђено иницијалном поставком апликације, те је посебна инжењерска пажња морала бити усмјерена на безбједносне импликације [33].

За контролу конкурентности користе се *CRDT* структуре података. Идентификовани су различити нивои конкурентности у самој структури документа, тако да се за редослијед ћелија, као и за садржај сваке ћелије понаособ, користи засебна, одговарајућа *CRDT* структура података [34], [35].

Сваки клијент држи *CRDT* структуру документа локално, примјењује измјене на њу одмах након што настану, као и када стигну са сервера. Сервер има своју копију *CRDT* структуре документа, примјењује долазеће измјене на њу и просљеђује их свим клијентима. Дакле, сервер има улогу посредника (и тачке перзистенције и контроле приступа), а не ауторитета за синхронизацију стања [36].

Рјешење се ослања на *Yjs* екосистем, стабилну имплементацију, испробану у продукцији која нуди исправне *CRDT* имплементације [34]. То укључује *Yjs* Јаваскрипт библиотека и *pycrdt* Пајтон омотач (енг. *bindings*) за за Раст пренос библиотеке *Yjs* [37]. Ова одлука софтверу доноси стабилност и једноставност. Цијена тога је што се, упркос архитектури са централним сервером, не користе олакшице које она пружа. То имплицира већу потрошњу рачунарских ресурса него што је неопходна, као и „закључавање” у исти *CRDT* екосистем алгоритама за све нивое конкурентности стања документа (и поредак ћелије и њихов садржај).

3.3 CoCalc

CoCalc је веб-платформа за колаборативно уређивање докумената и извршавање рачунарских програма. Има подршку за уређивање *Jupyter* биљежница у реалном времену, при чему се користи исти приступ као и за остале типове докумената [38].

Једино је рјешење за колаборацију на *Jupyter* биљежницама у реалном времену са јавно документованим алгоритмом [38]. Аутор идентификује три класична својства алгорита за синхронизацију:

- конвергенцију, да стање документа буде исто за све кориснике када се свачије измјене пропaгирају;
- очување узрочности, да операција коју је корисник добио прије настанка нове операције увијек буде прије ње;
- очување намјере корисника, да ефекат операције буде оно што је корисник хтио да постигне.

CoCalc испуњава прва два својства, али не и треће, допуштајући случајеве када операције имају нежељени ефекат. Ово је експлицитна одлука о дизајну која приоритизује једноставност алгоритма и имплементације у односу на гаранције коректности у сваком граничном случају [38].

За поредак ћелија и њихове метаподатке, сукоби се рјешавају употребом *LWW* принципа, док се за текстуални садржај ћелија примјењује посебан алгоритам заснован на операцијама из *diff-match-patch* (*DMP*) библиотеке, која садржи намјенске алгоритме за операције над текстом. Ради синхронизације текста, клијенти периодично израчунавају разлику између тренутног и претходно синхронизованог стања, на основу које се потом примјењује „закрпа” на стање документа [38], [39].

Одлука да се у потпуности заобиђу два стандардна приступа са *CRDT* и операционим трансформацијама у корист једноставнијег алгоритма са стриктно мањим гаранцијама исправности доноси питање употребљивости. У случајевима мале фокусираности избјегнута је комплексност рјешавања случајева који се заиста ријетко дешавају, али са већим бројем корисника и фокусиранијим обрасцима рада постоји реална потреба за већим гаранцима контроле конкурентности.

3.4 Google Docs

Google Docs представља де факто стандард за колаборативно уређивање текста у реалном времену, нудећи високу доступност, поузданост, контролу приступа и контролу конкурентности. Са прецизно имплементираним алгоритмом операционе трансформације за текст постиже колаборацију великог броја корисника без проблема са употребом рачунарских ресурса и без потешкоћа са конвергенцијом стања и очувањем намјере корисника.

Google Docs користи архитектуру са централним сервером, који је ауторитативан за стање документа и синхронизује га са клијентима [16]. Постојање централног сервера наметнуто је потребом за контролом приступа и гаранцијама за перзистентност стања, те се контрола конкурентности такође ослања на њу, чинећи систем кохезивним и робусним.

Операциона трансформација рјешава изазов измјена у истом дијелу документа без конфликта међу њима. Протокол за колаборацију осигурава да, када се више промјена дешава у исто вријеме, свака буде исправно спојена са осталима [16]. У наставку је дат преглед протокола за колаборацију на високом нивоу.

Ради постизања малог времена одзива, све измјене на клијентској страни се одмах примјењују на документ. Само једна измјена у једном тренутку се шаље на сервер, док остале постоје само локално и чекају њену потврду да би биле послате. Када са сервера стигне обавјештење о измјени другог корисника, она мора бити трансформисана у односу на локалне операције, да би могла бити исправно примијењена на локалну копију документа [16].

Сервер чува ауторитативну копију документа, као и историју операција измјене. Када нова операција стигне на сервер, она бива трансформисана у односу на све операције настале конкурентно њој, како би могла бити исправно примијењена на документ. Да би сервер знао које операције су конкурентне (настале

према истој верзији документа), прати се број ревизије и клијенти шаљу последњи број ревизије за који знају уз сваку операцију [16].

Овај протокол за колаборацију гарантује прецизност измјена и брзину, а главне предности укључују:

- ефикасност: преко мреже се шаље минимум информација потребних за опис промјене;
- константну комплексност колаборације: комплексност не расте са бројем клијената, јер сервер не познаје њихово стање;
- дистрибуираност: сервер и клијенти имају одвојене одговорности, тако да је терет алгорита распоређен између њих [16].

Пристап контроли конкурентности који користи *Google Docs* директно је инспирисао како се у овом раду третира текстуални садржај ћелија (видјети 6).

3.5 Figma

Софтвер за уређивање графичких докумената у реалном времену доноси изазове конкурентних измјена над комплексним стањем, изазване комплексношћу структуре документа. *Figma* ове изазове рјешава ослањајући се на рјешења инспирисана *CRDT* принципима, упрошћена гаранцијама које даје архитектура са централним сервером. Овај пристап има предност у односу на алгоритме засноване на операционој трансформацији јер је дефинисање исправне функције трансформације за комплексна стања са великим бројем различитих операција компликовано и подложно грешкама [7].

Конкретан примјер који се може позајмити у домен рачунарских биљежница је структура уређене листе са подршком за уметање, брисање и премјештање докумената. За рјешавање овог проблема, *Figma* користи технику фракционог индексирања описану у поглављу 2.1.5.3 [18].

У јуну 2025. *Figma* је увела функционалност слојева кода (енг. *code layers*) која захтијева синхронизацију великих текстуалних поља. За овај проблем одбацили су и *CRDT* структуру података и алгоритам операционе трансформације. *CRDT* пристап доносио је велики утршак меморије, јер чува јединствени идентификатор за сваки карактер. Операционе трансформације постају споре када постоји велики број конкурентних операција [40]. Ово је случај јер су блокови текста велики и подржано је офлајн уређивање, па повратак онлајн доводи до великог броја измјена над застарјелим документом. Компромисно рјешење које је одабрано је *Eg-walker* алгоритам из 2024. године који при рјешавању конфликта прави привремену *CRDT* структуру, а затим је одбацује, ослобађајући меморију [41].

3.6 Закључак поглавља

Преглед постојећих рјешења показује широк распон приступа проблему сарадње у реалном времену.

Алати из домена рачунарских биљежница проблем конкурентних измјена или заобилазе (закључавањем или *LWW* приступом), или га рјешавају уз компромисе (очувања намјере корисника или употребе рачунарских ресурса).

Системи изван овог домена, међутим, дали су конкретне градивне елементе за систем описан у овом раду.

Глава 4

Функционални и нефункционални захтјеви

Crab Collab је веб-апликација која омогућава да више корисника истовремено ради на једној *Jupyter* биљежници. Систем је реализован као доказ концепта, из чега слиједи критеријум по коме је одређен скуп захтјева. Обухваћена је свака функционалност неопходна да рјешење буде употребљиво, као и свака она која отвара засебан проблем контроле конкурентности или захтјева засебну стратегију имплементације. Изостављене су функционалности чије би увођење поновило већ приказану стратегију, не доносећи нови увид у истраживани проблем.

Ово поглавље даје преглед функционалних и нефункционалних захтјева постављених пред систем, као и ставки које су експлицитно изостављене из опсега рада. Захтјеви су изведени из особина групера у реалном времену изнесених у одјељку [2.1.1](#) и из анализе постојећих рјешења дате у поглављу [3](#).

4.1 Функционални захтјеви

4.1.1 Приступ сесији и идентификација учесника

Систем излаже једну дијељену биљежницу, над којом се одвија једна сесија. Корисник се придружује сесији отварањем веб-апликације и уносом свог имена. Како аутентификација није предвиђена, унесено име представља једини облик идентификације представљен осталим учесницима.

Систем сваком учеснику мора да додијели боју којом се разликује од осталих учесника у приказу присуства и позиције курсора.

При прикључењу, корисник мора да види почетно стање биљежнице које је идентично за све учеснике. Дакле, биљежница не мора бити празна уколико су корисници који су били раније присутни правили измјене.

4.1.2 Структура биљежнице

Биљежница мора бити приказана као вертикална листа ћелија, при чему сваку ћелију чине њен тип, текстуални садржај и, за ћелије кода, излаз посљедњег извршавања. Ћелија може бити ћелија кода или текстуална ћелија и њен тип се одређује при њеном настанку.

Систем мора да подржи следеће операције над листом ћелија:

- уметање: нова ћелија се појављује на жељеној позицији са празним садржајем и одређеног је типа;
- брисање: постојећа ћелија се трајно уклања;
- премјештање: постојећој ћелији се мијења позиција, без утицаја на садржај или стање извршавања.

Свака од наведених операција мора бити доступна сваком кориснику у сваком тренутку. Конкурентно извршавање структурних операција не смије нарушити интегритет листе. Конкретно, ниједна ћелија не смије бити нежељено изгубљена или удвостручена, а поредак ћелија мора остати исти за све учеснике.

Посебан случај представља конкурентно уметање на исту позицију. Систем мора да сачува обје унесене ћелије и једнозначно одреди њихов међусобни поредак, који је исти за све учеснике.

4.1.3 Уређивање текстуалног садржаја

Садржај сваке ћелије чини текст. Систем мора да омогући измјену текстуалног садржаја било које ћелије или било ког њеног дијела.

Систем мора да омогући да више корисника уређују садржај различитих ћелија, али и једне исте ћелије. Сваком кориснику мора бити омогућено уређивање текста у било ком тренутку (ћелија никада не смије бити „закључана”).

Садржај текстуалних ћелија мора бити приказан у форматираном облику, у складу са правилима маркдаун формата.

Не постоје ограничења за сам текстуални садржај.

4.1.4 Независност структурних и текстуалних измјена

Структурне измјене не смију да спречавају или на било који начин утичу на измјене текстуалног садржаја ћелија. Мора да важи и обратно, тј. текстуалне измјене не смију да спречавају или утичу на структурне.

4.1.5 Присуство и фокус

Систем мора да приказује листу учесника који су тренутно присутни у сесији.

За сваког учесника мора бити приказано на којој ћелији има фокус, као и позиција његовог курсора унутар садржаја те ћелије. Позиција курсора мора да буде приказана бојом додијељеном том учеснику.

Приказана позиција курсора мора да остане исправна и након измјена текстуалног садржаја, било тог корисника или неког другог.

4.1.6 Извршавање кода

Систем мора да омогући сваком учеснику да покрене извршавање ћелије кода. Код се извршава у једном дијељеном кернелу, тако да је стање кернела исто за све учеснике.

Кернел у једном тренутку може извршавати највише једну ћелију. Због тога систем мора да прихвата захтјеве за извршавањем у поретку којим пристижу и покреће извршавање једну за другом. Уколико више корисника конкурентно затражи извршавање, систем мора да одреди једнозначан поредак захтјева за извршавање. Уколико се ћелија извршава или постоји захтјев за њеним извршавањем на чекању, систем не смије прихватити нови захтјев за извршавање.

Ћелија се може налазити у једном од три стања извршавања:

- слободна,
- у реду за извршавање или
- у извршавању.

Стање извршавања сваке ћелије мора бити видљиво свим учесницима.

Излаз извршавања ћелије, укључујући стандардни излаз, стандардну грешку, резултат последњег израза и грешку при извршавању, мора бити приказан свим учесницима. Излаз је дио стања ћелије, тако да увијек мора бити везан за покренуту ћелију и мора бити приказан корисницима који се прикључе након извршавања.

Из комбинације конкурентног уређивања и извршавања кода произилазе гранични случајеви чије понашање систем мора да дефинише:

- Уколико се садржај ћелије промијени након упућеног захтјева за извршавање, мора да буде извршен садржај који је ћелија имала у тренутку упућивања захтјева.
- Уколико ћелија буде обрисана у току њеног извршавања или док се њено извршавање налази у реду чекања, излаз мора бити одбачен. Само извршавање, које може имати бочне ефекте на стање кернела, не смије бити прекинуто.

4.2 Нефункционални захтјеви

4.2.1 Временска ограничења

Систем припада класи групвера у реалном времену, из чега слиједе захтјеви над двије временске величине дефинисане у одјељку [2.1.1](#).

Вријеме одзива мора бити довољно мало да измјене које корисник уноси дјелују тренутно.

Вријеме обавјештења мора бити упоредиво са временом одзива. Доња граница времена обавјештења одређена је мрежним кашњењем. Систем не смије да уноси видљиво додатно кашњење осим тога.

4.2.2 Конзистентност

Систем мора да задовољи три својства алгоритама за синхронизацију дефинисана у одјељку [3.3](#):

- конвергенција: након пропагирања свих измјена, стање документа мора бити идентично за све учеснике;
- очување узрочности: узрочно повезане измјене морају бити примијењене истим редослиједом код свих учесника;
- очување намјере корисника: ефекат примијењене измјене корисника мора да одговара намјери учесника који ју је унио.

Нужан изузетак за очување намјере корисника представља конкурентно помјерање исте ћелије на више различитих позиција од стране више корисника. У овом случају систем мора да испуни намјеру само једног корисника.

Конзистентност има приоритет над ефикасношћу, тако да рјешења која умањују гаранције конзистентности ради боље употребе рачунарских ресурса нису прихватљива.

4.2.3 Ефикасност употребе рачунарских ресурса

Меморија коју систем заузима треба да буде сразмјерна величини садржаја документа, а не броју унесених измјена или броју учесника.

Комплексност обраде измјена не смије да расте са бројем истовремених учесника [\[16\]](#).

Ови захтјеви односе се на све дијелове система (сервер и клијенте).

4.2.4 Модуларност и проширивост

Механизми контроле конкурентности за различите дијелове стања документа морају бити раздвојени, тако да механизам за један дио стања може бити замијењен без измјена у остатку система.

Функционалности изостављене из тренутног опсега не смију да захтијевају измјену архитектуре при накнадном увођењу.

4.2.5 Употребљивост

Кориснички интерфејс треба да буде упоредив са интерфејсима постојећих апликација за *Jupyter* биљежнице. Не смије бити већих утицаја на употребљивост због захтјева за сарадњу у реалном времену.

4.3 Ставке ван опсега

Ставке наведене у овом одјељку изостављене су из опсега рада из три различита разлога:

- ради очувања фокуса на проблем контроле конкурентности,
- због тога што њихово увођење не доноси нови увид у истраживачки проблем или
- због сложености која превазилази обим рада.

Ставке су груписане према разлогу изостављања, јер он образлаже зашто изостављање дате групе не умањује допринос рада.

4.3.1 Изостављено ради очувања фокуса

Ставке у овој групи неопходне су за продукцијску употребу система, али су ортогоналне проблему контроле конкурентности. Оне укључују:

- управљање сесијама: систем излаже једну биљежницу, без подршке за више докумената или сесија;
- постојаност стања: стање сесије чува се искључиво у меморији сервера и губи се при његовом гашењу;
- аутентификација и ауторизација: не постоји пријава корисника нити ограничавање приступа документу;
- скалабилност: систем се извршава као један процес и не предвиђа расподјелу оптерећења на више инстанци;
- безбједност: изузев *НМАС* потписа порука које захтијева *Jupyter* протокол, безбједносни аспекти нису разматрани;
- офлајн уређивање: активна мрежна веза услов је за рад са документом.

Ниједна од наведених ставки не захтијева измјену архитектуре при накнадном увођењу, што је постављено као захтјев у одјелку [4.2.4](#).

4.3.2 Изостављено због недостатка доприноса истраживању

Ставке у овом одјелку дијелом су подржане *Jupyter* протоколом, али њихова имплементација не подразумева проблем контроле конкурентности који није покривен постојећим функционалностима. Уз сваку ставку наведен је и начин на који би била уведена, јер он представља основ за њено изостављање.

Опсези селекције Приказ присуства обухвата позицију курсора, али не и опсег означеног текста. Опсег чине двије позиције, на које се без измјена примјењује трансформација позиције курсора описана у поглављу [6](#), па увођење не захтијева нови механизам.

Богати излази Подржан је само текстуални излаз, док слике, графици, *HTML* садржај и интерактивни елементи нису обрађени. Увођење би захтијевало проширење модела излаза на *MIME* пакете и одговарајући приказ на клијенту, уз исти механизам пропагирања који се користи за текстуални излаз.

Стандардни улаз Захтјеви кернела за кориснички улаз, попут позива функције `input()`, нису подржани. Захтјев би био прослијеђен учеснику који је покренуо извршавање, док би осталима био приказан као дио излаза ћелије, чиме се задржава једнак поглед на стање за све учеснике.

Операције над кернелом Поновно покретање, прекид извршавања и гашење кернела нису изложени корисничком интерфејсу. Оне се преносе *Control* каналом, а њихов ефекат на стање кернела пропагирао би се истим механизмом као и стање извршавања ћелија. Поновно покретање додатно захтијева пражњење реда за извршавање.

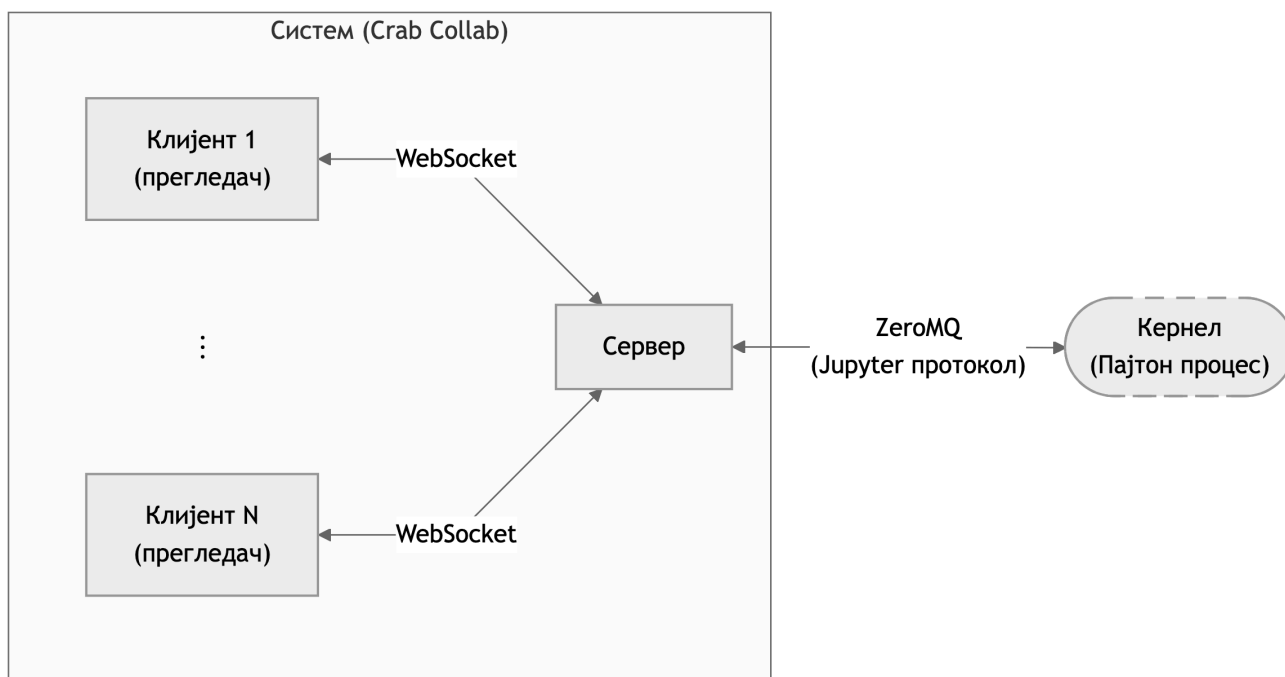
Више кернела и језика Подржан је један кернел за програмски језик Пајтон. *Jupyter* протокол је независан од језика, па би увођење захтијевало откривање доступних кернела и њихово покретање, без измјена у обради измјена документа.

4.3.3 Изостављено због сложености проблема

Поништавање и понављање измјена (енг. *undo* и *redo*) у групверу у реалном времену подразумева надоградњу алгоритама синхронизације и пажљиво разматрање жељене семантике за примјену на документе сложене структуре. Познато је да је ово један од најсложенијих проблема у домену операционих трансформација [\[14\]](#), што говори о генералној сложености овог проблема. Увођење ових операција представља посебан истраживачки проблем, а не проширење постојећег система. Стога је искључено из опсега рада.

У овом поглављу представљена је архитектура апликације *Crab Collab* на високом нивоу, без залажења у имплементационе детаље. Дата су и образложења битних архитектонских одлука.

5.1 Преглед архитектуре



Слика 2: Дијаграм архитектуре система

На слици 2 приказан је дијаграм компоненти система. Систем чине три врсте процеса:

- клијент, који се извршава у веб-прегледачу и представља кориснички интерфејс према биљежници;
- сервер, који одржава стање сесије, комуницира са клијентима и управља кернелом;
- кернел, засебан процес који извршава код.

Може да постоји произвољан број клијената. Сваки клијент одржава једну *WebSocket* везу према серверу и сва комуникација тече преко ње. Клијенти не комуницирају међу собом.

Сервер је одговоран за управљање кернел процесом и комуницира са њим преко *Jupyter* протокола.

Сервер је ауторитативан за стање сесије. Једини одлучује о томе које измјене су прихваћене и којим редослиједом. Сваки клијент чува локално стање ради приказа кориснику и имплементације алгоритама синхронизације.

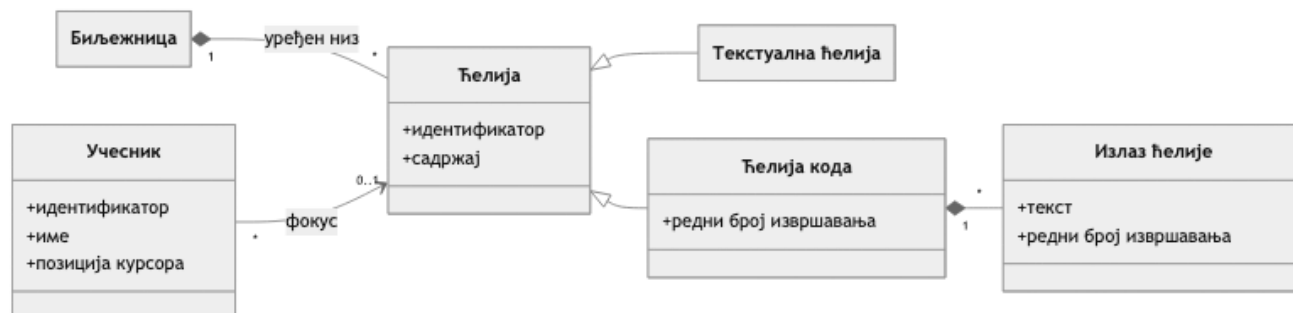
5.2 Модел стања

5.2.1 Модел података

Стање сесије чине документ биљежнице и подаци о учесницима.

Биљежница је уређен низ ћелија. Свака ћелија има јединствени идентификатор, текстуални садржај и тип. Ћелије кода додатно садрже редни број извршавања и излаз последњег извршавања.

Учесник је описан јединственим идентификатором, именом које уноси при придруживању, ћелијом на којој има фокус и позицијом курсора унутар садржаја те ћелије. Боја којом се учесник приказује није дио стања него се детерминистички изводи из јединственог идентификатора корисника.



Слика 3: Модел података

На слици 3 приказан је упрошћен дијаграм класа који приказује основне елементе модела података и односе између њих.

5.2.2 Домени стања

Потребе корисника и структура модела података захтијевају да се различити дијелови стања сесије могу мијењати конкурентно, и то користећи различите и независне стратегије контроле конкурентности. Зато је стање сесије декомпоновано на пет домена, од којих се сваки синхронизује независно. Уколико је потребно направити снимак комплетног стања, онаквог какво је описано у претходном одјелу (нпр. за приказ у корисничком интерфејсу или постојаност стања), то је увијек могуће учинити спајањем дијелова из различитих домена.

У систему *Crab Collab* издвојени су следећи домени стања:

Поредак ћелија Уређена листа над којом се врши уметање, брисање и премјештање.

Садржај ћелија Текст једне ћелије мијења више учесника истовремено. Садржај сваке ћелије представља независан дио стања.

Позиције курсора Позиције курсора морају да се помјерају измјеном садржаја ћелије. Дакле, ово стање зависи од садржаја ћелије, али не и од осталих домена.

Стање извршавања Захтјеви за извршавање независни су од осталих операција. Конкурентност се овдје рјешава серијализацијом захтјева.

Присуство учесника Придруживање сесији и њено напуштање. Догађаји су независни од свих осталих па конфликти нису ни могући.

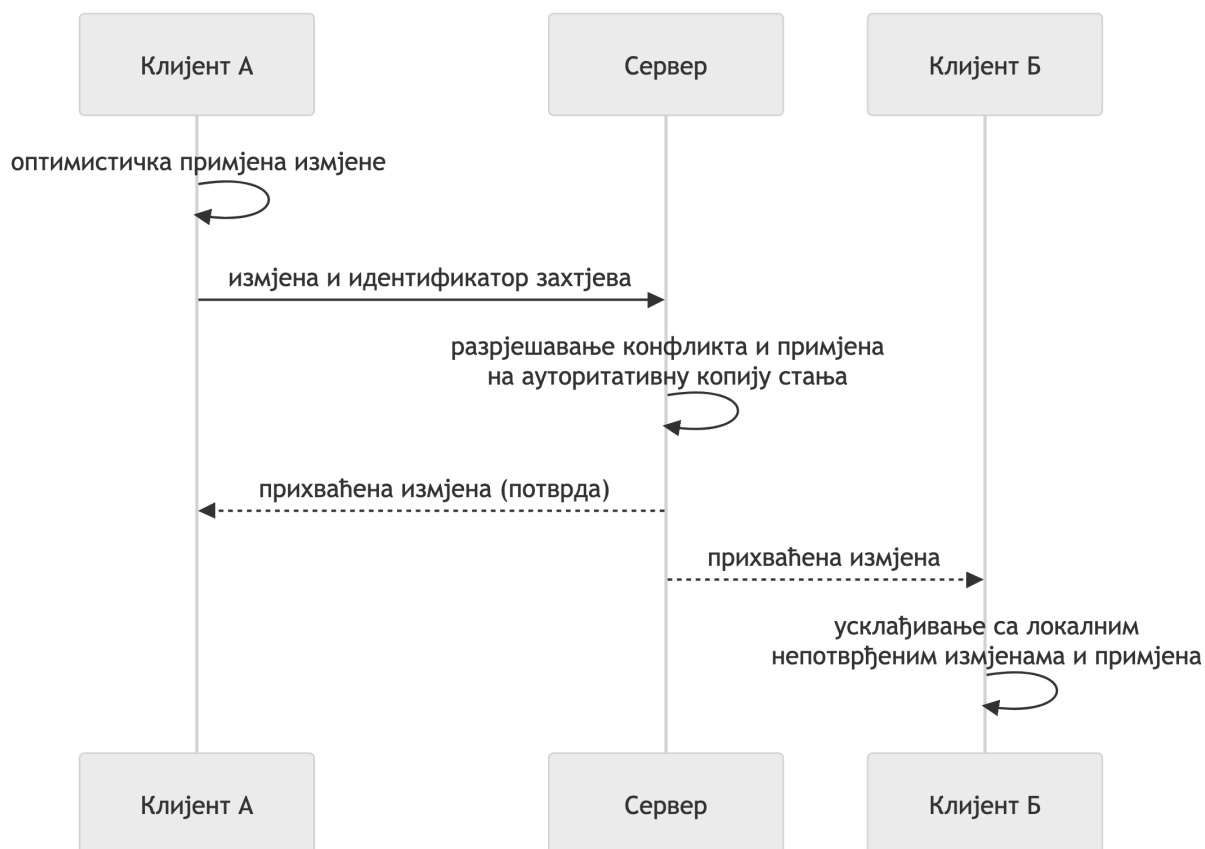
Подјела на домене такође је основ за захтјев за модуларношћу из поглавља 4.2.4, јер омогућава да се стратегије контроле конкурентности мијењају за један домен независно од других.

5.3 Модел комуникације

Домени стања описани у претходном одјелу се разликују по природи конфликта, али свака измјена стања пролази исти образац размјене:

1. Клијент примјењује измјену на своју локалну копију стања, чиме се задовољава захтјев за кратким временом одзива.
2. Клијент шаље измјену серверу и памти је као непотврђену.
3. Сервер прима измјену, рјешава евентуални конфликт са конкурентним измјенама, примјењује измјену на ауторитативно стање и утврђује њено мјесто у поретку прихваћених измјена.
4. Сервер шаље измјену свим корисницима, укључујући и аутора.

5. Учесници примају поруку и примјењују је на своје локално стање. Аутор само потврђује већ примјењену измјену. Овај корак подразумијева рјешавање конфликта са локалним измјенама које нису потврђене са сервера.



Слика 4: Ток једне измјене кроз систем

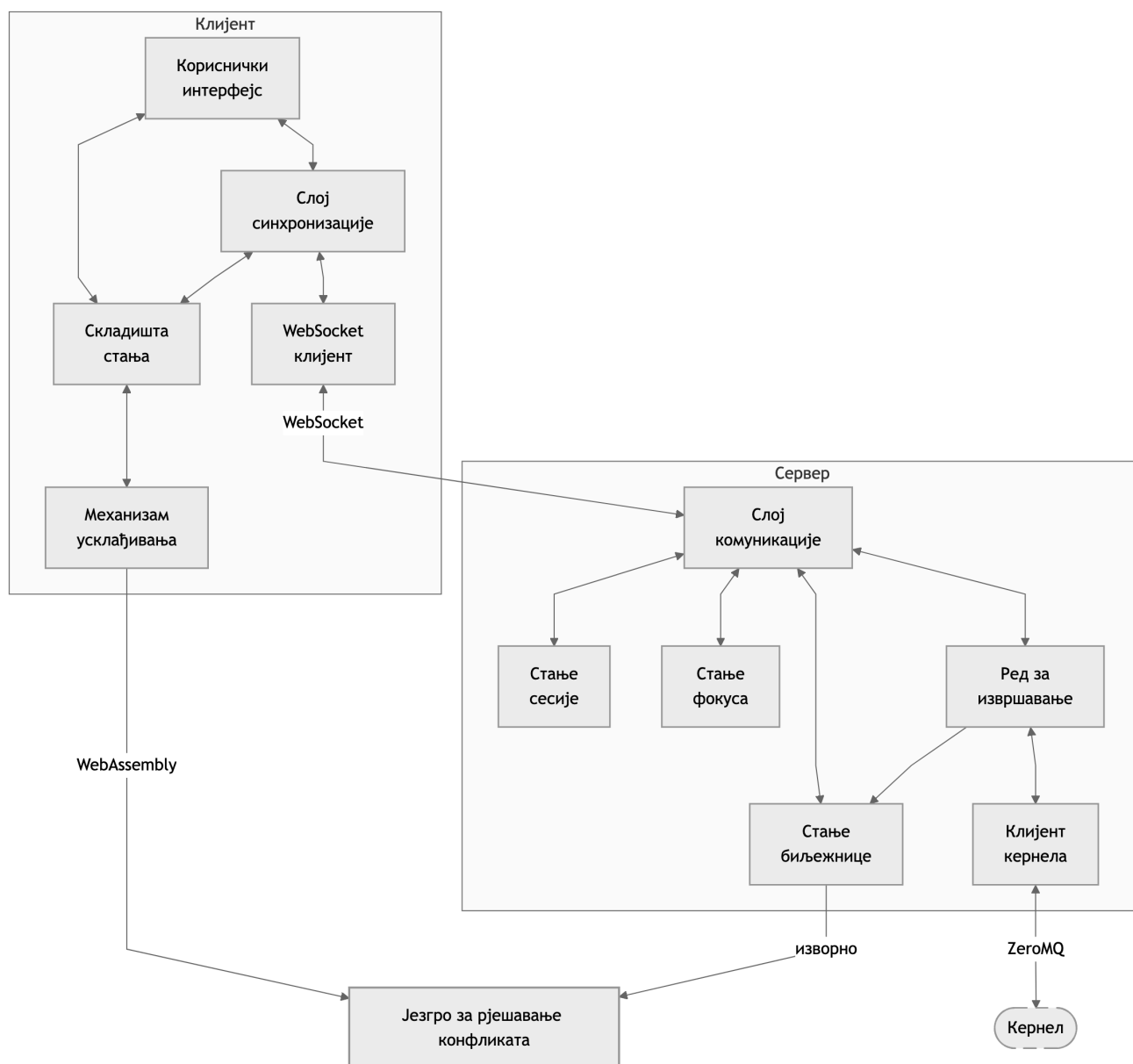
На слици 4 приказан је редослијед корака за измјену коју уноси један учесник, од њеног настанка до пропагирања у поглед осталих учесника.

Уз сваку послату измјену сервер шаље идентификатор учесника који ју је затражио и идентификатор захтјева који је тај учесник додијелио при слању. Ова два податка довољна су да сваки клијент разликује сопствену измјену од туђе и да непотврђену измјену повеже са њеном потврдом.

Уколико сервер не може да прихвати измјену, о томе обавјештава искључиво њеног аутора, који тада поништава своју локалну измјену. Овај случај се јавља код операција над ћелијом која је у међувремену обрисана и код захтјева за извршавање, чија ћелија није слободна.

Описани образац не зависи од домена стања над којим се врши измјена, нити од механизма за рјешавање конфликта.

5.4 Структура компоненти



Слика 5: Структура серверске и клијентске компоненте

У овом одјељку дат је преглед модула унутар серверске и клијентске компоненте. Унутар сваке компоненте модули су функционално раздвојени ради јасне расподеле одговорности, али се извршавају у истом процесу. На слици 5 је преглед свих модула са везама између њих.

5.4.1 Сервер

Серверска компонента подијељена је на модуле према одговорностима:

Слој комуникације Управља *WebSocket* везама, обавља серијализацију и десеријализацију порука и просљеђује поруке одговарајућим модулима.

Стање сесије Одржава списак повезаних учесника заједно са стањем потребним слоју комуникације да шаље поруке учесницима.

Стање фокуса Одржава информације о ћелији на којој сваки корисник има фокус, као и позицији његовог курсора.

Стање биљежнице Чува ауторитативно стање документа. Приступ овом стању је апстрахован интерфејсом, тако да се имплементација може замијенити. Модул је задужен да исправно управља конкурентношћу стања и делегира механизме рјешавања конфликта намјенским модулима који их имплементирају.

Ред за извршавање Прати захтјеве за извршавање кода. Служи као медијатор између слојева за комуникацију са клијентима и кернелом, уз очување интегритета реда захтјева.

Клијент кернела Имплементира *Jupyter* протокол за комуникацију са кернелом и излаже га модулу за ред за извршавање као једноставан *API* (енг. *Application Programming Interface*).

Језгро за рјешавање конфликта По један модул за сваку одвојену стратегију. Садрже механизме, тј. алгоритме и структуре података за рјешавање конфликта. Дизајнирани су као библиотеке кода који може да користи модул за управљање стањем.

5.4.2 Клијент

Клијентска компонента подијељена је на слојеве:

Кориснички интерфејс Приказује биљежницу и присуство учесника, а унос корисника просљеђује као операције другим модулима.

Складишта стања Локална копија стања у меморији. Складишта садрже само податке и једноставне операције над њима.

Механизам усклађивања Одржава оптимистички примјене и непотврђене измјене и обавља њихово усклађивање са измјенама које пристигну са сервера.

Слој синхронизације Преводи поруке примљене са сервера у позиве механизма за усклађивање и складишта, а корисничке акције у поруке које се шаљу серверу. Овдје се налази и управљање временским аспектима слања, попут одлагања и ограничења учестаности слања измјена текста.

WebSocket клијент Одржава везу са сервером, серијализује и десеријализује поруке.

Језгро за рјешавање конфликта Исти модули које користи и сервер, преведени у *WebAssembly* формат.

5.4.3 Дијељено језгро за рјешавање конфликта

Језгро за рјешавање конфликта преведено је двапут из истог изворног кода: једном изворно, за сервер, и једном у *WebAssembly* за клијента. Осим очигледне предности непонављања кода, то има значај за гаранцију исправности. Одржавање двије имплементације истог алгоритма у два различита програмска језика увело би реалну опасност да имплементације дивергирају у погледу значења операција, што би довело до неисправног функционисања система. Дијељење исте имплементације осигурава да је немогуће да се то догоди.

5.5 Преглед усвојених механизма

Од пет домена стања издвојених у одјељку 5.2.2, за два је потребно размотрити механизме за контролу конкурентности са посебном пажњом јер захтијевају механизам за спајање конкурентних измјена. То важи за поредак ћелија и њихов текстуални садржај.

За поредак ћелија усвојено је фракционо индексирање, по узору на рјешење које *Figma* користи за уређене листе.

За садржај ћелије усвојена је операциона трансформација у централизовану поставку, по узору на *Google Docs*.

Позиције курсора не захтијевају посебан механизам, него се директно изводе помоћу механизма коришћења за текст.

Код преостала два домена конкурентност се не рјешава спајањем. Захтјеви за извршавање серијализују се у ред на серверу. Догађаји придруживања сесији и напуштања исте не зависе једни од других нити од другог стања и могу се извршавати слободно.

Ово поглавље описује како су механизми усвојени у поглављу 5.5 имплементирани у програмском коду. Њихов преглед организован је према доменима стања уведеним у поглављу 5.2.2.

Дио механизма преузет је из постојећих библиотека, што софтверу пружа стабилност провјерених имплементација. Ријеч је о алгоритмима чија је имплементација подложна грешкама, а од чије исправности зависи исправност читавог система, па је употреба провјерених имплементација свјесна одлука. Сопствени допринос представља имплементација комплетних механизма која се само ослања на ове библиотеке.

Исјечци кода дати у наставку скраћени су ради прегледности. Изостављени су опширни дијелови кода који не доприносе излагању.

6.1 Организација изворног кода

6.1.1 Сервер

Серверски дио система имплементиран је у програмском језику Раст. Организован је као Cargo радни простор састављен од шест сандука.

Табела 1: Сандуци и њихове одговорности

Сандук	Одговорност
server	WebSocket везе, протокол и оркестрација
notebook	модел биљежнице и управљање стањем
ot	операције над текстом
crdt	уређена листа са фракционим индексима
execution_queue	праћење захтјева за извршавање
kernel	Jupyter клијент преко ZeroMQ

Списак сандука дат је у табели 1. Сандуци ot и crdt чине језгро за рјешавање конфликта. Они не зависе ни од server-а, ни од notebook-а, не посједују доменско знање нити доменску логику. То их чини погодним за употребу и на серверу и на клијенту, унутар одговарајућих алгоритама за дати контекст.

```
#[cfg_attr(feature = "wasm", wasm_bindgen)]
pub struct TextOperation(OperationSeq);
```

Листинг 1: wasm_bindgen атрибут иза обиљежја wasm

Превођење у WebAssembly управљано је обиљежјем (енг. *feature*) wasm, као у примјеру у листингу 1. При превођењу за сервер атрибут не постоји и ради се о обичном Раст типу. При превођењу за клијента, помоћу библиотеке wasm_bindgen, исти тип добија омотач који је могуће позивати из Јаваскрипта.

6.1.2 Клијент

Клијентски дио система је засебан пројекат имплементиран у програмском језику Тајпскрипт. Кориснички интерфејс израђен је помоћу библиотеке React. За приказ уређивача текста користи се библиотека Monaco.

Локална копија стања чува се у меморији помоћу складишта имплементираних помоћу библиотеке Zustand. Измјене над стањем описане су као да мијењају податке на лицу мјеста, а библиотека Immer од сваке

такве измјене прави нову непромјенљиву верзију стања. Тако функције које имплементирају механизме усклађивања остају обичне функције над подацима, и не зависе од библиотеке за складиште.

Структурно, пројекат прати подјелу на слојеве уведену у одјељку 5.4.2, при чему сваком слоју одговара засебан дио изворноґ кода:

- кориснички интерфејс: *React* компоненте у директоријуму `components`, груписане по приказу ћелије, биљежнице и присуства учесника;
- складишта стања: `notebookStore`, `sessionStore` и `userStore` у директоријуму `stores`;
- механизам усклађивања: модул `notebookEngine`, скуп функција над стањем биљежнице;
- слој синхронизације: куке (енг. *hooks*) `useNotebookSync`, `useTextSync` и `useFocusSync`, по једна за сваки домен стања који захтијева усклађивање;
- *WebSocket* клијент: кука `useWebSocket`;
- језгро за рјешавање конфликта: модули `ot` и `crdt` преведени у *WebAssembly* формат.

6.2 Апстракција стања биљежнице

6.2.1 Особина `NotebookStateHolder`

Приступ ауторитативном стању биљежнице апстрахован је особином (енг. *trait*) `NotebookStateHolder`. Функције особине групишу се управо по доменима стања:

- структурне измјене: `apply_cell_operation`;
- текстуалне измјене: `apply_text_operation`, `get_cell_version`;
- позиције курсора: `rebase_cursor_position`;
- стање извршавања: `append_cell_output`, `clear_cell_output`, `set_cell_execution_number`;
- читање стања: `get_notebook`, `get_notebook_state`.

Овако одабран скуп функција омогућава раздвајање стања по доменима у интерном представљању ради обезбјеђивања својстава конкурентности, уз подршку за читање комплетноґ стања или неког његовоґ дијела.

Како су методе асинхроне, а особина се користи у контексту који захтијева динамички диспеч, употребљен је макро `#[async_trait::async_trait]`.

Ова апстракција омогућава одабир имплементације на једном мјесту, при покретању сервера, јер се чува у структури `AppState` као `Arc<dyn NotebookStateHolder>`. Остатак сервера користи само ову апстракцију и не мора да буде свјестан детаља имплементације механизма контроле конкурентности.

6.2.2 Гранулација закључавања

```
pub struct ConcurrentStateHolder {
    cells: Arc<DashMap<CellId, CellState>>,
    order: RwLock<OrderingState>,
}

struct CellState {
    cell: Cell,
    version: u64,
    history_base_version: u64,
    operation_history: VecDeque<TextOperationResult>,
}

struct OrderingState {
    version: u64,
    cell_order: FractionalList<CellId>,
}

#[async_trait::async_trait]
impl NotebookStateHolder for ConcurrentStateHolder {
    // ...
}
```

Листинг 2: Структура `ConcurrentStateHolder` која имплементира особину `NotebookStateHolder`

Конкретна имплементација која је коришћена носи назив `ConcurrentStateHolder` и приказана је у листингу 2. Она користи независне механизме закључавања за поредак ћелија и њихов садржај.

Поредак ћелија чува се под једном бравом (енг. *lock*) типа `RwLock` која серијализује операције које мијењају стање, али дозвољава више конкурентних читача.

Тип `DashMap` [42] је мапа подијељена на сегменте тако да се сваки сегмент независно закључава. Број сегмената сразмјеран је броју доступних процесора, што је и мјера која одређује колико операција уопште може бити истовремено у току. Повећање броја преко те мјере не смањује сукобе, а троши додатну меморију.

Оно што ова структура заиста обезбјеђује јесте да измјене података различитих ћелија у уобичајеном случају користе различите браве. Очигледнија алтернатива, `RwLock<HashMap<CellId, Cell>>`, изгледа једноставније, али би у том случају операције над независним ћелијама увијек долазиле у сукоб, јер би биле серијализоване иза једне браве.

Операције уметања и брисања ћелије су једине које користе оба стања:

- Уметање ћелије не може да буде конкурентно операцијама над истом, јер она прије те операције не постоји.
- Брисање ћелије може да буде конкурентно операцијама над истом, али пошто ћелија након те операције више неће постојати, стање ће бити идентично и ако операције над ћелијом буду примјењене прије брисања и ако их одбацимо након брисања. То значи да није потребно уводити додатно закључавање у овом случају.

6.2.3 Спречавање узајамног блокирања

Да би се избјегло узајамно блокирање (енг. *deadlock*), потребно је увијек користити досљедан редослијед закључавања браве. Конкретно, обје операције прво закључавају поредак, па тек онда приступају мапи ћелија.

До узајамног блокирања може доћи и унутар саме DashMap-е уколико држимо више референци на ћелије у истом сегменту. Овај случај се избегава тако што се никада не држи референца на двије ћелије у истој путањи кода.

6.2.4 Конзистентност снимка стања

Снимак стања цијеле биљежнице чита ћелије једну по једну, закључавајући по један сегмент мапе у сваком тренутку. Структура биљежнице при томе остаје конзистентна, јер се брава над поретком држи током цијелог читања. Садржаји ћелија, међутим, потичу из различитих тренутака, и текстуална измјена може да се догоди између два корака читања.

Ово не представља проблем. Свака ћелија има сопствени простор верзија и конзистентност њеног стања не зависи од осталих ћелија. Клијент који прими овакав снимак наставља рад са сваком ћелијом од верзије која јој је придружена.

Алтернатива би била истовремено закључавање свих ћелија, што би зауставило свако уређивање током прављења снимка.

6.2.5 Независност домена стања

Оваква структура закључавања сама по себи испуњава захтјев за независношћу структурних и текстуалних измјена из одјелка 4.1.4. Независност се заснива на томе да операције закључавају само оно стање коме приступају.

Из тога слиједи подјела одговорности у остатку имплементације. Структуре `OrderingState` и `CellState` чувају стање које је већ заштићено бравама, тако да је њихов једини задатак рјешавање конфликта унутар свог домена.

6.3 Поредак ћелија

6.3.1 Фракциони индекси

У одјелку 2.1.5.3 уведени су фракциони индекси као реални бројеви. Реални бројеви се користе као индекси листе и, како се увијек може наћи неки реалан број између било која два друга, тако увијек можемо додјелом фракционог индекса смјестити елемент на било које мјесто у листи.

За имплементацију фракционих индекса, систем се ослања на библиотеку `fractional_index` [43]. Из практичних разлога, индекси нису представљени бројевима са покретним зарезом. Разлог је што такво представљање нуди ограничену прецизност, па се може догодити да не можемо да представимо ниједан реалан број између друга два.

```
pub struct FractionalIndex(Vec<u8>);
```

Листинг 3: Дефиниција типа `FractionalIndex`

Умјесто тога, бројеви су представљени низом бајтова (листинг 3), гдје сваки бајт представља једну цифру у бројном систему са базом 256. Подразумијева се да су цифре иза зареза, тако да додавањем произвољног броја бајтова добијамо произвољну прецизност.

Библиотека нуди три конструктора за фракциони индекс:

- `new_before`: прави индекс прије датог;
- `new_after`: прави индекс након датог;
- `new_between`: прави индекс између дата два индекса.

У најгорем случају, који настаје узастопним уметањем на исто мјесто, индекс расте за по један бајт при уметању. За биљежницу са десетинама ћелија и повременим уметањима ово ограничење нема практичан значај.

6.3.2 Уређена листа

Структура `FractionalList` користи тип `FractionalIndex` за имплицитно одржавање редослиједа елемената произвољног типа. Нуди функције за уметање, брисање и премјештање елемената, као и за приступ тренутном поретку елемената, па је једноставна за коришћење.

```
pub struct FractionalList<Id: ElementId> {
    by_id: HashMap<Id, FractionalIndex>,
    by_index: BTreeMap<FractionalIndex, Id>,
}

impl<Id: ElementId> FractionalList<Id> {
    // ...

    pub fn get_ordered(&self) -> impl Iterator<Item = &Id> {
        self.by_index.values()
    }

    pub fn insert_at(&mut self, id: Id, index: &FractionalIndex) -> FractionalIndex {
        let new_index = self.get_real_index(index);
        self.by_id.insert(id.clone(), new_index.clone());
        self.by_index.insert(new_index.clone(), id);
        new_index
    }

    pub fn delete(&mut self, id: Id) {
        if let Some(idx) = self.by_id.remove(&id) {
            self.by_index.remove(&idx);
        }
    }

    pub fn move_to(&mut self, id: Id, index: &FractionalIndex) -> FractionalIndex {
        let new_index = self.get_real_index(index);
        if let Some(idx) = self.by_id.get_mut(&id) {
            self.by_index.remove(idx);
            self.by_index.insert(new_index.clone(), id);
            *idx = new_index.clone();
        }
        new_index
    }

    fn get_real_index(&self, index: &FractionalIndex) -> FractionalIndex {
        if !self.by_index.contains_key(index) {
            return index.clone();
        }
        match self.by_index.range((Excluded(index), Unbounded)).next() {
            Some((idx, _)) => FractionalIndex::new_between(index, idx).unwrap(),
            None => FractionalIndex::new_after(index),
        }
    }
}
```

Листинг 4: Дефиниција типа `FractionalList`

Структура интерно одржава двије мапе:

- `HashMap`: одржава индексе за сваки елемент и нуди приступ у константном времену;
- `BTreeMap`: одржава поредак индекса, али и омогућава проналажење елемента по индексу у логаритамском времену.

На листингу 4 можемо видјети да се ефикасна имплементација жељених операција помоћу ове двије структуре своди на свега неколико операција над двије мапе.

Кључно питање које треба ријешити је шта се дешава када операција покушава да елементу додијели индекс који је већ заузет. Одговор је да се може једноставно заузети слободан индекс након елемента који већ држи жељени, али прије његовог сљедбеника. За то служи помоћна функција `get_real_index` која користи структуру `BTreeMap` за проналажење индекса сљедбеника.

Ово је валидна стратегија јер сервер држи ауторитативно стање, па може да одреди редослијед којим се операције примјењују и коначни индекс елемента, као што је објашњено у наредном одјељку.

6.3.3 Протокол за колаборацију

И сервер и клијент чувају своју копију `FractionalList` структуре података у којој се чувају јединствени идентификатори ћелија. Из ње се у било ком тренутку може извести јединствен поредак ћелија. Клијент овај поредак користи за приказ на екран, а сервер при генерисању снимка стања.

Структурне измјене преносе се порукама `cell_insert`, `cell_delete` и `cell_move`. Поруке увијек референцирају елементе по њиховим јединственим идентификаторима, што их чини недвосмисленим. Поруке за уметање и премјештање садрже фракционе индексе у хексадекадном формату. Овај формат је безбједан за `JSON` поруке и лексикографско уређење чува поредак самих индекса.

За синхронизацију стања користи се модел комуникације описан у поглављу 5.3. Не постоји ограничење на број операција које клијент може да пошаље без пријема потврде са сервера. Детаљи о операцијама укључују сљедеће специфичности:

1. При стварању операције клијент мора да генерише фракциони индекс.
2. Сервер задржава право да промијени индекс који је додијелио клијент уколико дође до конфликта, односно конкурентне операције другог клијента која користи исти индекс. Индекс који одреди сервер шаље се у одговору.
3. Клијент операције других корисника просто примјењује на локално стање, а потврде својих информација прихвата.

```
impl<Id: ElementId> FractionalList<Id> {
    // ...

    pub fn move_to_strict(&mut self, id: Id, index: &FractionalIndex) -> FractionalIndex {
        if let Some(conflict_id) = self.by_index.get(index) {
            if *conflict_id == id {
                return index.clone();
            }
            self.move_to(conflict_id.clone(), &self.get_real_index(index));
        }
        self.move_to(id, index)
    }
}
```

Листинг 5: Имплементација функције `move_to_strict` за ауторитативно помјерање

При пријему поруке са сервера, може се десити да постоји локална операција са истим индексом као ново-пристигла операција. У том случају, клијент мора промијенити локалне операције тако да свака операција има јединствен индекс и поредак ћелија буде одржан. Разликујемо два случаја:

- Уколико је аутор новопристигле операције други корисник, она не постоји локално. Тада можемо уклонити ћелију која прави проблем из поретка, применијенити новопристиглу операцију, а затим покушати да уметнемо стару ћелију назад на исту позицију. Алгоритам за уметање ће аутоматски одредити нову позицију на којој ће се наћи локални елемент.

- Када стигне потврда операције која већ постоји, можемо ћелију о којој се ради једноставно премјестити на нови индекс. Уколико већ постоји нека ћелија на том индексу, морамо је помјерити тако да поредак остане исти. То можемо постићи употребом функције `move_to_strict` приказане у листингу 5, посебно намијењене за овај случај.

Овај алгоритам за локално рјешавање конфликта ради исправно јер су испуњени сљедећи услови:

- Серијализација операција на серверу и протокол за комуникацију (*WebSocket*) гарантују да операције стижу клијентима у истом редослиједу у ком су примјењене на серверу.
- `FractionalList` увијек рјешава конфликте око истог индекса на исти начин: помјерајући удесно ћелију коју мијења операција која долази касније у јединственом редослиједу операција.
- Све операције се шаљу свим клијентима.

Случај који је интересантан за размотрити је када више корисника конкурентно помјери исту ћелију. Претходно објашњени алгоритам гарантује да ће резултујућа позиција бити одређена по *LWW* принципу, што је семантички исправно понашање.

Вриједи напоменути још да јединствене идентификаторе ћелија генеришу клијенти. Да би се осигурало да су глобално јединствени, користи се *UUID* (енг. *Universally Unique Identifier*). Иако постоје начини да се генеришу јединствени идентификатори који заузимају мање меморије, употребљен је овај ради једноставности имплементације. Број ћелија у једном документу је типично довољно мали да је меморија коју заузимају идентификатори безначајна.

6.4 Текстуални садржај ћелија

6.4.1 Операциона трансформација

Операција над текстом је низ основних корака над индексом у низу знакова који га помјерају од првог до посљедњег знака. Постоје три типа основних корака:

- задржи (енг. *retain*): прескаче задати број знакова, остављајући их непромијењеним;
- уметни (енг. *insert*): умеће дати текст и оставља индекс на његовом крају;
- обриши (енг. *delete*): уклања задати број знакова.

Табела 2: Функције алгоритама операционе трансформације за текст

Псеудо потпис	Намјена
<code>transform(a: Op, b: Op)</code> <code>-> (a1: Op, b1: Op)</code>	Функција трансформације. Трансформише двије операције једну у односу на другу и враћа обе операције трансформисане.
<code>compose(a: Op, b: Op) -> Op</code>	Враћа операцију чија је примјена еквивалентна примјени двије дате операције секвенцијално.
<code>apply(a: Op, s: str) -> str</code>	Примјењује дату операцију на текст.

Алгоритми који користе овај модел операције за рад са текстом склони су грешкама при имплементацији, тако да су узете постојеће имплементације из библиотеке *operational-transform* [44]. Систем користи алгоритме који су имплементирани у функцијама чији је концептуални преглед дат у табели 2. У стварности, ово су методе над структуром *OperationSeq*, којом библиотека представља једну операцију.

6.4.2 Протокол за колаборацију

За синхронизацију стања користи се модел комуникације описан у поглављу 5.3. У овом одјелу описани су начини за рјешавање конфликта на серверској и клијентској страни.

Текст сваке ћелије третира се независно од других и на серверу и на клијентима. У наставку се подразумијева да разматрамо протокол за садржај једне ћелије.

Свака ћелија има своју верзију, која је суштински верзија њеног садржаја. Верзија је неозначени цијели број који се повећава за један након сваке измјене. Уз сваку операцију клијент шаље основну верзију: најновију верзију ћелије за коју он зна. Уз сваки одговор, сервер шаље најновију верзију ћелије на серверу, тако да клијенти могу да ажурирају основне верзије са своје операције.

Клијент и сервер размјењују један тип поруке за пренос измјена, `text_edit`, који, осим података о аутору и верзији, садржи текстуалну операцију. Операција је серијализована у компактан низ компатибилан са *JSON* форматом и представља низ основних операција (задржи, уметни, обриши).

Гаранције о редослиједу порука које дају закључавање стања на серверској страни и *WebSocket* протокол за комуникацију чине исправним алгоритме за синхронизацију стања на серверској и клијентској страни описане у наставку.

6.4.2.1 Стање на серверу

Сервер чува историју свих операција и тренутну верзију ћелије. Када операција пристигне са клијентске стране, операције које су њој конкурентне су добиле верзију већу од основне верзије коју је клијент послао. Сервер ради следеће:

1. Трансформише операцију у односу на све конкурентне операције.
2. Трансформисану операцију примјењује на ауторитативну копију текста.
3. Повећава верзију ћелије за један.
4. Додаје операцију у историју.
5. Шаље трансформисану операцију, заједно са новом верзијом, свим клијентима.

Чињеница да сервер чува цијелу историју операција има за посљедицу да употреба меморије расте са бројем измјена и не зависи само од тренутног стања, што је у супротности са захтјевом из 4.2.3. Да би се ово ријешило уводи се максимална величина историје коју сервер памти. Када се капацитет попуни, избацује се најстарија операција.

Најстарија верзија коју сервер мора да чува може се извести из основних верзија које шаљу активни клијенти. Овај приступ није реализован ради максималне фокусираности на главни предмет рада, очувањем једноставности имплементације.

Ако се деси да клијент пошаље поруку са основном верзијом мањом од најстарије у историји, сервер не може да ријешити конфликт операционом трансформацијом. Зато се уводи нова порука, `cell_resync`, која клијенту шаље најновије стање садржаја ћелије.

Капацитет историје бира се у односу на учестаност слања описану у наставку. Подразумијевани капацитет од 1000 операција, уз максимални размак између слања порука од 100 милисекунди (видјети 6.4.3), одговара периоду куцања од око минут и по. Како систем захтијева сталну мрежну везу са сервером, то би требало бити довољно да покрије типичне случајеве коришћења.

```

impl CellState {
    fn apply_text_operation(
        &mut self,
        base_cell_version: u64,
        operation: TextOperation,
        origin_id: OriginId,
    ) -> Result<TextOperationResult, NotebookError> {
        if base_cell_version < self.history_base_version {
            return Err(/* ... */);
        }

        let start = (base_cell_version - self.history_base_version) as usize;
        let mut real_op = operation;
        for op_result in self.operation_history.iter().skip(start) {
            let transform_result = transform(&op_result.operation, &real_op)?;
            real_op = transform_result.b_prime();
        }
        self.cell.content = apply(&real_op, &self.cell.content)?;

        self.version += 1;

        let result = TextOperationResult { /* ... */ };

        self.operation_history.push_back(result.clone());
        if self.operation_history.len() > CELL_OPERATION_HISTORY_CAP {
            self.operation_history.pop_front();
            self.history_base_version += 1;
        }

        Ok(result)
    }
    //...
}

```

Листинг 6: Алгоритам трансформисања операције кроз историју

Листинг 6 приказује кључни дио алгоритма на серверској страни који трансформише операцију кроз историју.

6.4.2.2 Стање на клијенту

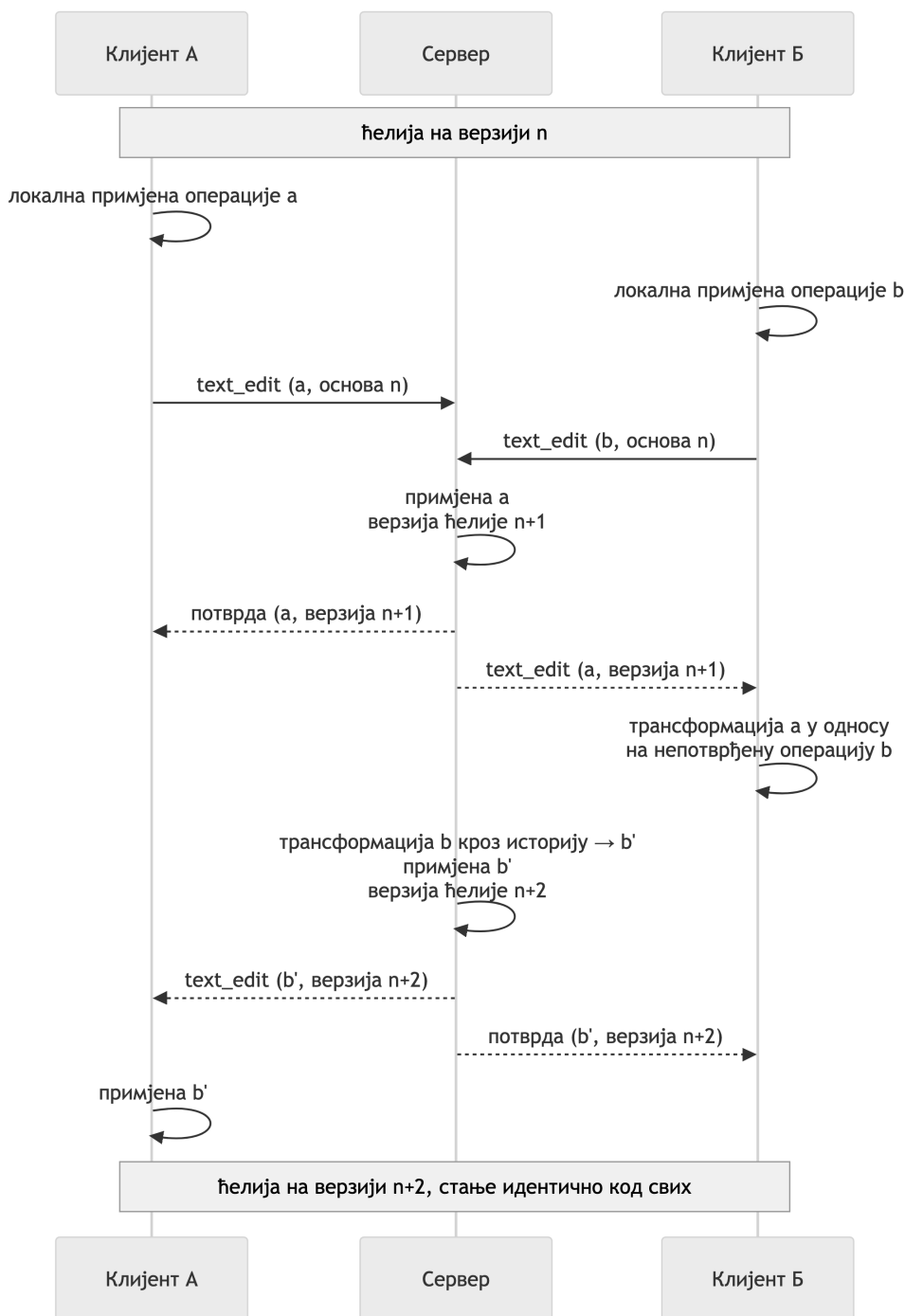
Операције које клијентска апликација оптимистички приказује кориснику, а нису још увијек послате на сервер зовемо „операције на чекању”. Операције које су послате на сервер, али још увијек нису добиле своју потврду зовемо непотврђене. Ово су једине операције које клијент памти. Видимо да клијент и сервер чувају различито стање у меморији, тако да је одговорност алгоритма дистрибуирана између њих.

У било ком тренутку, највише једна операција може да буде непотврђена. Операције које настају док непотврђена операција чека потврду остају на чекању. Да се не би нагомилало много ситних операција на чекању, оне се спајају у једну операцију функцијом `compose`. Тако постоји највише једна операција на чекању, чиме се смањује укупан број послатих порука, што побољшава перформансе. Када стигне потврда са сервера, операција на чекању може да се пошаље и промовише у непотврђену.

Када са сервера стигне операција од другог аутора, она је сигурно трансформисана у односу на све њој конкурентне операције на серверској страни. Међутим, локалне операције сервер сигурно није обрадио прије ње. То значи да новопристигла операција мора да се трансформише у односу на непотврђену и операцију на чекању да би могла исправно да се примијени на локалну копију стања. Редослијед транс-

формација је битан. Трансформација у односу на непотврђену операцију мора да буде прва, јер је она прва и настала.

6.4.2.3 Примјер комуникације



Слика 6: Дијаграм секвенце измјена текста

На слици 6 дат је дијаграм секвенце који приказује један примјер колаборације над текстом. Демонстрирана је стратегија рјешавања конкурентних измјена.

6.4.3 Временско понашање слања

Текстуалне операције типично настају у кратким временским размацама. Зато су уведена два временска ограничења на њихово слање са клијентске стране.

Прво је одлагање након мировања (енг. *debounce*) које шаље операције тек када корисник престане да куца. Друго је ограничење учестаности (енг. *throttle*), које осигурава да се измјене шаљу и у току непрекидног куцања.

Оба су потребна. Само одлагање би, при непрекидном куцању, слање одгађало неограничено и нарушило вријеме обавјештења. Само ограничење учестаности би, након сваке паузе, слало још једну операцију без потребе.

6.4.4 Кодирање позиција

Уредник текста и модел документа у прегледачу броје позиције у UTF-16 кодним јединицама, док их операције броје у кодним тачкама стандарда Unicode. Двије мјере се разилазе за сваки знак изван основне вишејезичне равни, попут емоџија.

Правило је стога једноставно: све што прелази границу ка језгру преводи се у кодне тачке, а унутар језгра се барата искључиво њима. Превођење се ослања на то да `Array.from` пролази кроз низ знакова по кодним тачкама, држећи сурогатни пар на окупу.

6.5 Позиције курсора

Стање фокуса за једног корисника састоји се од ћелије на којој има фокус и позиције курсора унутар текстуалног садржаја те ћелије. Стање фокуса је независно између учесника тако да може слободно да се мијења без посебног механизма за синхронизацију.

Клијент шаље поруку `change_focus` која садржи најновије стање фокуса. Стање се ажурира на серверу и просљеђује свим осталим клијентима.

Слање података о фокусу подложно је временској контроли, јер може да се мијења јако често. Имплементирано је одлагање након мировања (енг. *debounce*).

6.5.1 Промјене у тексту

Позиција курсора зависи од текстуалног садржаја. Промјеном текста мијењају се позиције курсора за све кориснике који имају курсор у неком дијелу тог текста, било да су аутори измјена или не.

Типичан случај је да стање курсора треба послати након измјене текста. Да би се избјегла ситуација да се ажурирање фокуса пошаље прије текстуалне измјене која га је узроковала, уведен је додатни кратки период од тренутка настанка измјене до тренутка када је дозвољено ажурирати фокус.

Сам механизам којим се позиције курсора мијењају у односу на промјене текста директно је изведен из алгорита операционе трансформације који се користи за текстуалне операције. Имплементиран је у функцији `transform_position`, која пролази кроз основне операције од којих је сачињена измјена да трансформише позицију на основу текстуалне операције.

Превођење позиције пролази кроз исти пресјек историје као и текстуалне операције. Механизам за курсоре стога није само изведен из механизма за текст, него користи и његову структуру података. Клијент исти поступак спроводи локално, у односу на своју непотврђену операцију и операције на чекању, прије приказа.

Уметање тачно на позицију курсора је двосмислено. Ако је текст унио сам власник курсора, курсор мора остати иза унесеног текста. Ако га је унио неко други, курсор мора остати на мјесту. Одговор даје идентификатор аутора уписан уз сваку операцију у историји, чијим поређењем се одлучује како ће се позиција понијети. Ово је најмањи могући примјер очувања намјере корисника.

```
impl CellState {
    fn rebase_position(
        &self,
        base_cell_version: u64,
        position: usize,
        origin_id: OriginId,
    ) -> usize {
        let start = base_cell_version.saturating_sub(self.history_base_version) as usize;
        let mut pos = position;
        for op_result in self.operation_history.iter().skip(start) {
            pos = transform_position(pos, &op_result.operation, op_result.origin_id == origin_id);
        }
        pos
    }
    //...
}
```

Листинг 7: Трансформисање позиције курсора кроз историју текстуалних операција

6.5.2 Гаранције конзистентности

Промјена фокуса има утицај само на тренутно стање, и то само једног корисника. Ниједна историјска промјена фокуса, осим посљедње, нема утицаја на тренутно стање. Овако кратак домет утицаја промјена фокуса у комбинацији са учесталашћу промјена фокуса, која је велика у поређењу са осталим операцијама, доводи до закључка да одржање конзистентности не мора да има главни приоритет за овај домен стања.

Конкретно, случај недостатка историје за трансформацију позиције курсора, не третира се стриктно као грешка, као што је случај са текстуалним операцијама од чије тачности зависи конвергенција стања садржаја ћелије. С обзиром на то да ће већ сљедећим притиском тастера фокус бити промијењен, овакви случајеви могу бити игнорисани. Ово показује да третман стања фокуса са слабијим гаранцијама конзистентности има директну предност једноставности кода.

Такође, у конкретном случају, дозволио је да се избјегне закључавање текстуалног садржаја за измјене у случају прављења снимка тренутног стања фокуса. Ово значи да је постигнуто директно побољшање перформанси на штету гаранција конзистентности. Исти компромис јавља се и код снимка садржаја ћелија, о чему је била ријеч у одјелку [6.2.4](#).

Овај примјер јасно говори да стратегија подјеле стања на домene доноси једну додатну предност. Могуће је поставити различите захтјеве за конзистентност за различите домene стања. Ово је битно јер често начин руковања подацима од стране корисника директно одређује које гаранције конзистентности су потребне, а он може да буде другачији за различите домene стања.

6.6 Извршавање кода

Овај домен разликује се од претходних по томе што нема спајања конкурентних измјена. Кернел може да извршава само један захтјев у једном тренутку, па се они могу само поредати. Контрола конкурентности се своди на питање гдје се успоставља поредак захтјева и ко га одржава.

Одговор зависи од начина интеграције са кернелом, тако да су у овом поглављу дати детаљи имплементације клијента кернела и додатног кода за вођење евиденције о захтјевима.

6.6.1 Повезивање са кернелом

Сервер је задужен да покрене кернел као посебан процес. Прије покретања уписује датотеку повезивања, у којој наводи адресу, портове за пет канала протокола, тајни кључ и шему потписивања. Портове додјељује оперативни систем, привременим везивањем на порт нула. Кључ је насумично генерисан *UUID*. Кернел се затим покреће наредбом `python3 -m ipykernel_launcher -f <путања>`, тако да путању до датотеке повезивања добије као аргумент.

```

let mut shell = zeromq::DealerSocket::new();
shell.connect(&connection_file.shell_endpoint()).await?;
let (shell_send, shell_recv) = shell.split();

let mut iopub = zeromq::SubSocket::new();
iopub.connect(&connection_file.iopub_endpoint()).await?;
iopub.subscribe("").await?; // Subscribe to all topics

let mut control = zeromq::DealerSocket::new();
control.connect(&connection_file.control_endpoint()).await?;

let mut hb = zeromq::ReqSocket::new();
hb.connect(&connection_file.hb_endpoint()).await?;

```

Листинг 8: Успостављање везе са *Jupyter* кернелом преко *ZeroMQ* сокета

Након покретања успостављају се сокети описани у одјелку 2.2.4, што можемо видјети на листингу 8. За рад са *ZeroMQ* сокетима користи се библиотека *zeromq* из пројекта *zmq.rs* [45], која имплементира потребне примитиве у Расту. Канали *Control* и *Heartbeat* су отворени и опслужени, тако да у случају проширења система на операције на нивоу кернела нису потребне измјене кода за конекцију, који је осјетљив.

Слој за комуникацију нуди асинхрони интерфејс. Функција `execute_code` шаље поруку `execute_request` на *Shell* каналу и одмах враћа идентификатор те поруке, без чекања на било какав одговор. Два засебна *tokio* задатка слушају на каналима *Shell* и *IOPub*, преводе примљене поруке и шаљу их даље интерним каналом.

```

pub enum OutputEvent {
    Stream { name: StreamName, text: String },
    ExecuteResult { execution_count: u32, data: OutputData },
    Error { ename: String, evalue: String, traceback: Vec<String> },
    Status { state: KernelState },
    ExecutionFinished { execution_count: u32, status: ExecutionStatus },
}

```

Листинг 9: Тип `OutputEvent`, граница сандука `kernel`

Тип `OutputEvent` приказан на листингу 9 је граница сандука `kernel`. Иза границе остају оквири, потписи, заглавља и подјела на канале, а испред ње стоји неколико догађаја који просто описују ток извршавања. Сандук `kernel` при томе не познаје домен биљежница, који припада слоју изнад, чиме граница повучена у поглављу 5.4.1 остаје видљива у коду.

6.6.2 Ред захтјева за извршавање

Захтјеви постављени пред овај домен своде се на три ставке:

- кернел у једном тренутку извршава највише једну ћелију,
- конкурентни захтјеви морају добити једнозначан поредак,
- ћелија која се већ извршава или чека на извршавање не смије прихватити нови захтјев.

Прве двије ставке су испуњене самим *Jupyter* протоколом. *Shell* канал јесте *ZeroMQ* сокет, дакле асинхрона апстракција реда порука, а кернел са њега преузима једну по једну поруку, редом којим су пристигле.

Одатле слиједи одлука да сервер не уводи сопствени ред. Његово увођење било би редундантно јер би се захтјеви пребацивали из једног реда у други. Тиме би се увело ново стање које дефинише исти поредак који је утврђен и простом употребом *ZeroMQ* сокета.

```
pub struct Execution {
    pub message_id: String,
    pub cell_id: CellId,
    pub requester_id: Uuid,
    pub timestamp: DateTime<Utc>,
    pub status: ExecutionStatus,
}

pub struct ExecutionQueue {
    jupyter_client: JupyterClient,
    pending_executions: Arc<RwLock<HashMap<CellId, Execution>>>,
}

impl ExecutionQueue {
    // ...
    pub async fn execute(&self, cell: Cell, requester_id: Uuid) -> Result<(), ExecutionError> {
        {
            let mut pending = self.pending_executions.write().await;
            if pending.contains_key(&cell.id) {
                return Err(ExecutionError::AlreadyQueued(cell.id));
            }

            pending.insert(cell.id, Execution { /* ... */ });
        }

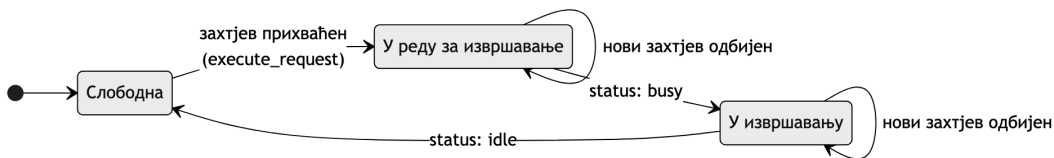
        match self.jupyter_client.execute_code(&cell.content).await {
            Ok(msg_id) => {
                if let Some(execution) = self.pending_executions.write().await.get_mut(&cell.id) {
                    execution.message_id = msg_id;
                }
                Ok(())
            }
            Err(err) => {
                self.pending_executions.write().await.remove(&cell.id);
                Err(err.into())
            }
        }
    }
}
```

Листинг 10: Вођење евиденције о захтјевима за извршавање у сандуку execution_queue

Стање које је потребно да сервер чува у меморији јесте веза између поруке протокола и ћелије на коју се она односи. Та веза се чува у мапи чији је кључ идентификатор ћелије. Помоћу те мапе можемо одбити захтјеве за извршавање ћелија које су већ у евиденцији, једноставном провјером постојања кључа, као што је приказано у листингу 10.

Поредак се успоставља у тренутку уписа поруке на сокет. Приступ страни за слање заштићен је закључавањем, па два конкурентна захтјева нужно бивају уписана један за другим. То је уједно и редослијед којим их кернел извршава и редослијед којим сервер о њима обавјештава клијенте, тако да сви учесници виде исти поредак.

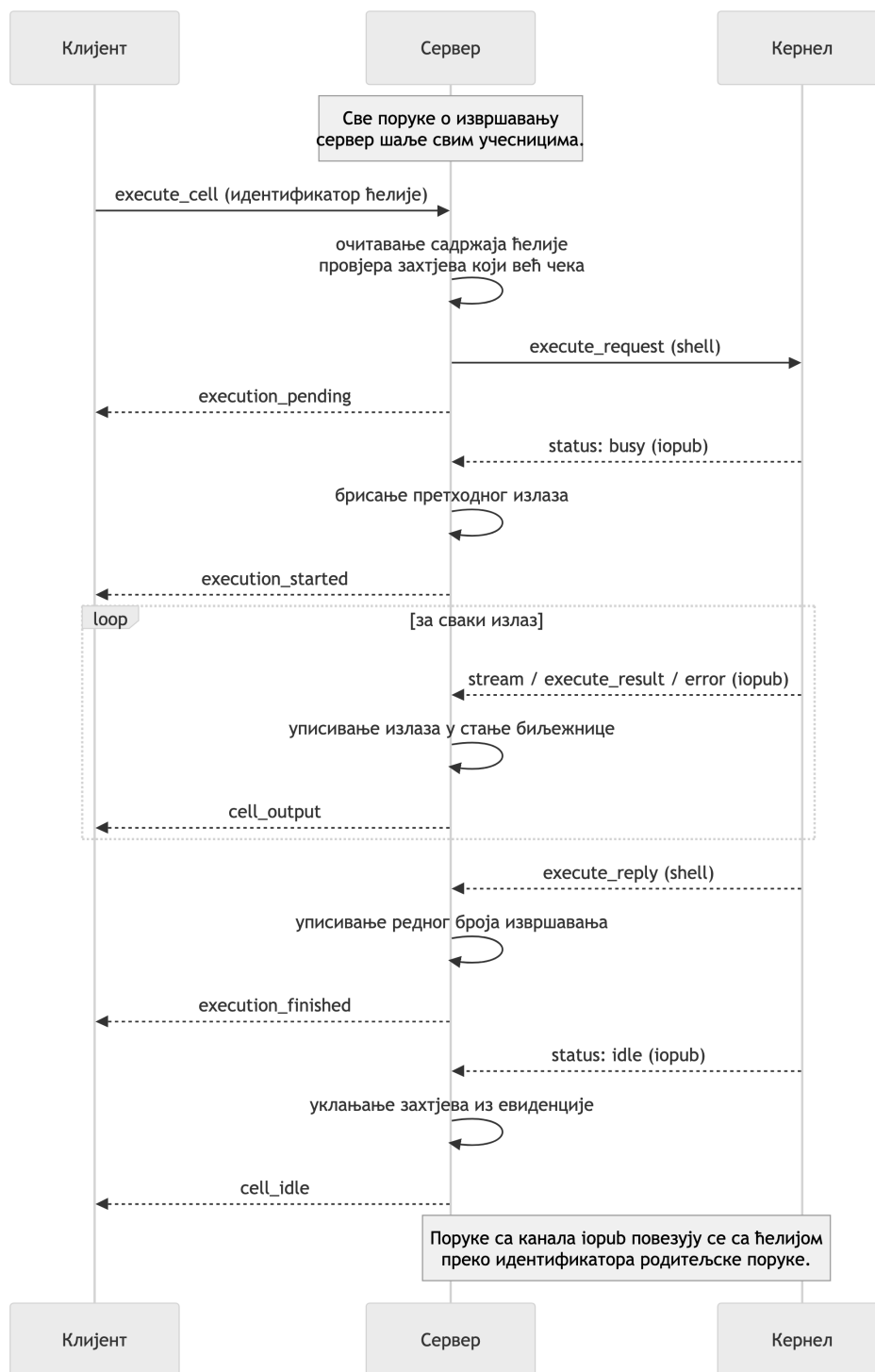
Посебан *tokio* задатак слуша на поруке са кернела. Свака порука коју кернел пошаље као посљедицу извршавања носи заглавље изворне поруке, па се ћелија добија тражењем уноса чији се идентификатор поруке поклапа са идентификатором родитељске поруке.



Слика 7: Дијаграм стања извршавања ћелије, са прелазима изазваним порукама протокола

Стање извршавања ћелије мијења се према порукама кернела и има три вриједности описане у поглављу 4.1.6: слободна, у реду за извршавање или у извршавању. Захтјев се не уклања из евиденције поруком `execute_reply` која означава крај извршавања, него поруком `status: idle`, која долази после и значи да је кернел заправо слободан за извршавање. Прелаз стања извршавања ћелије у зависности од порука кернела приказан је на дијаграму стања на слици 7.

6.6.3 Ток извршавања



Слика 8: Дијаграм секвенце једног извршавања

На слици 8 може се наћи дијаграм секвенце који описује комплетан ток једног извршавања, са размјенама порука на релацијама клијент-сервер и сервер-кернел.

Захтјев учесника преноси се поруком `execute_cell`, која носи само идентификатор ћелије. Сервер из стања биљежнице чита садржај те ћелије и предаје га кернелу. Тиме је испуњен захтјев да се изврши садржај који је ћелија имала у тренутку упућивања захтева.

О току извршавања сервер обавјештава све учеснике порукама `execution_pending`, `execution_started`, `cell_output`, `execution_finished` и `cell_idle`. Стање извршавања и излаз једнаки су за све учеснике, па се аутор овдје не издваја, него сви клијенти једнако третирају поруку.

Излаз извршавања се уписује у ауторитативно стање ћелије прије него што буде послат. Овај редослијед је битан јер омогућава да учесници који се придруже сесији у току извршавања ћелије имају исправан поглед на излазе, добивши снимак комплетног стања.

Претходни излаз ћелије брише се на поруку `status: busy`, а не при прихватању захтјева. Тако излаз претходног извршавања остаје видљив све док се захтјев за извршавање налази у стању чекања.

Изузев почетног захтјева за извршавање, све промјене стања у овом домену настају од стране кернела. Зато нема потребе за оптимистичким примјенама нити чувањем непотврђеног стања на клијенту, што су измјене модела комуникације из одјелка [5.3](#).

Одбијени захтјев не производи обавјештење. Клијент дугме за покретање држи онемогућеним док ћелија није слободна, па до одбијања долази само када два учесника конкурентно покрену исту ћелију. Провјера на серверу је коначна, а сувишни захтјев се одбацује без посљедица по стање. Будући да ће се ћелија извршити, очувана је намјера оба учесника.

6.6.4 Гранични случајеви

Из комбинације конкурентног уређивања и извршавања произилазе два гранична случаја. Поглавље [4.1.6](#) прописује жељено понашање за њих.

Први гранични случај је измјена садржаја ћелије након упућеног захтјева, ријешен снимком садржаја описаним у претходном одјелку.

Други се односи на брисање ћелије која је у извршавању или чека на извршавање. Како се захтјев шаље кернелу одмах на пријему, немогуће га је зауставити у случају брисања ћелије, што је и жељено понашање. Излаз се одбацује јер код који жели да га забиљежи у ауторитативно стање не може да нађе ћелију која више не постоји.

Стратегија која би такође била оправдана у доменском смислу је да се захтјеви на чекању поништавају уколико се ћелија обрише. Осим што би оваква интерпретација била подложна нејасноћама (нпр. захтјев за брисање ћелије не стигне до сервера прије него што она пређе у извршавање), сама имплементација би била захтјевнија. У овом случају би ипак било потребно да сервер држи посебан ред извршавања у меморији, чије би постојање било оправдано једино овим захтјевом.

6.7 Присуство учесника

Придруживање и напуштање сесије преносе се порукама `join` и `leave`. При придруживању сервер новом учеснику шаље потпуно стање: биљежницу, верзије по ћелији, фракционе индексе ћелија, захтјеве за извршавање који чекају, списак присутних учесника и додијељени идентификатор.

Боја учесника изводи се детерминистички из његовог идентификатора на сваком клијенту, па се никада не преноси нити је потребно усагласити је.

Овај домен нема механизам за рјешавање конфликта, нема верзије и нема трансформације. Његово издвајање као засебног домена управо је оно што је ту чињеницу учинило видљивом.

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак слика

Слика 1	Архитектура комуникације између <i>Jupyter</i> фронтенда и кернела [21] ²	7
Слика 2	Дијаграм архитектуре система	19
Слика 3	Модел података	20
Слика 4	Ток једне измјене кроз систем	21
Слика 5	Структура серверске и клијентске компоненте	22
Слика 6	Дијаграм секвенце измјена текста	34
Слика 7	Дијаграм стања извршавања ћелије, са прелазима изазваним порукама протокола	39
Слика 8	Дијаграм секвенце једног извршавања	40

²Извор: *Jupyter Development Team*. Слика је доступна под условима *Modified BSD* лиценце. Доступно на: <https://jupyter-client.readthedocs.io/en/latest/>. Приступљено: 24. августа 2026. год.

Списак листинга

Листинг 1	<code>wasm_bindgen</code> атрибут иза обиљежја <code>wasm</code>	25
Листинг 2	Структура <code>ConcurrentStateHolder</code> која имплементира особину <code>NotebookStateHolder</code>	27
Листинг 3	Дефиниција типа <code>FractionalIndex</code>	28
Листинг 4	Дефиниција типа <code>FractionalList</code>	29
Листинг 5	Имплементација функције <code>move_to_strict</code> за ауторитативно помјерање	30
Листинг 6	Алгоритам трансформисања операције кроз историју	33
Листинг 7	Трансформисање позиције курсора кроз историју текстуалних операција	36
Листинг 8	Успостављање везе са <i>Jupyter</i> кернелом преко <i>ZeroMQ</i> сокета	37
Листинг 9	Тип <code>OutputEvent</code> , граница сандука <code>kernel</code>	37
Листинг 10	Вођење евиденције о захтјевима за извршавање у сандуку <code>execution_queue</code>	38

Списак табела

Табела 1	Сандуци и њихове одговорности	25
Табела 2	Функције алгоритама операционе трансформације за текст	31

Списак коришћених скраћеница

Скраћеница	Опис
НПР	TODO: Додати скраћенице

Списак коришћених појмова

Појам	Објашњење
Појам	TODO: Додати коришћене појмове

Биографија

Никола Јоловић рођен је 26. децембра 2002. године у Зворнику.

Одрастао је у Бијељини, гдје је завршио Основну школу Кнез Иво од Семберије и средњу Техничку школу Михајло Пупин. У основној и средњој школи истакао се на такмичењима из физике, математике и информатике, те је проглашаван за најбољег ђака у генерацији у оба степеника школовања.

Након завршене средње школе, одлази у Нови Сад, гдје уписује студије на Факултету техничких наука, смјер Софтверско инжењерство и информационе технологије. Током студија, истиче се, како високим оцјенама, тако и учешћем и dobrим резултатима на хакатонима.

Као студент стиче искуство из индустрије радећи стручну праксу у *Codolis*-у, бавећи се *full-stack* развојем софтвера. Након 2 године у *Codolis*-у, прелази на позицију стажисте у *JetBrains*-у, радећи на проблему скалирања *CI* агената за *TeamCity*.

- [1] S. Lau, I. Drosos, J. M. Markel, and P. J. Guo, “The Design Space of Computational Notebooks: An Analysis of 60 Systems in Academia and Industry,” in *2020 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2020, pp. 1–11.
- [2] T. Kluyver *et al.*, “Jupyter Notebooks – a publishing format for reproducible computational workflows,” in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds., IOS Press, 2016, pp. 87–90.
- [3] H. Shen, “Interactive notebooks: Sharing the code,” *Nature*, vol. 515, no. 7525, p. 152, 2014.
- [4] A. Rule, A. Tabard, and J. D. Hollan, “Exploration and explanation in computational notebooks,” in *Proceedings of the 2018 CHI conference on human factors in computing systems*, 2018, pp. 1–12.
- [5] A. Davies, F. Hooley, P. Causey-Freeman, I. Eleftheriou, and G. Moulton, “Using interactive digital notebooks for bioscience and informatics education,” *PLOS Computational Biology*, vol. 16, no. 11, p. e1008326, 2020.
- [6] J. Day-Richter, “What’s different about the new Google Docs: Working together, even apart.” Accessed: August 22, 2026. [Online]. Доступно на https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs_21.html
- [7] E. Wallace, “How Figma’s multiplayer technology works.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.figma.com/blog/how-figmas-multiplayer-technology-works/>
- [8] A. Y. Wang, A. Mittal, C. Brooks, and S. Oney, “How data scientists use computational notebooks for real-time collaboration,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. CSCW, pp. 1–30, 2019.
- [9] C. A. Ellis and S. J. Gibbs, “Concurrency control in groupware systems,” in *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, 1989, pp. 399–407.
- [10] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.
- [11] H. T. Kung and J. T. Robinson, “On optimistic methods for concurrency control,” *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.
- [12] C. A. Ellis, S. J. Gibbs, and G. L. Rein, “Groupware: some issues and experiences,” *Communications of the ACM*, vol. 34, no. 1, pp. 38–58, 1991.
- [13] J. Day-Richter, “What’s different about the new Google Docs: Conflict resolution.” Accessed: August 22, 2026. [Online]. Доступно на https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs_22.html
- [14] C. Sun and C. Ellis, “Operational transformation in real-time group editors: issues, algorithms, and achievements,” in *Proceedings of the 1998 ACM Conference on Computer Supported Cooperative Work*, 1998, pp. 59–68.
- [15] A. Randolph, H. Boucheneb, A. Imine, and A. Quintero, “On Consistency of Operational Transformation Approach,” 2013.

-
- [16] J. Day-Richter, “What’s different about the new Google Docs: Making collaboration fast.” Accessed: August 22, 2026. [Online]. Доступно на <https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs.html>
- [17] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Symposium on Self-Stabilizing Systems*, 2011, pp. 386–400.
- [18] E. Wallace, “Realtime Editing of Ordered Sequences.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.figma.com/blog/realtime-editing-of-ordered-sequences/>
- [19] D. Sun, C. Sun, A. Ng, and W. Cai, “Real differences between OT and CRDT in correctness and complexity for consistency maintenance in co-editors,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 4, no. CSCW1, pp. 1–30, 2020.
- [20] P. Hintjens, *ZeroMQ: Messaging for Many Applications*. O’Reilly Media, 2013.
- [21] Jupyter Development Team, “Messaging in Jupyter.” Accessed: August 24, 2026. [Online]. Доступно на <http://jupyter-client.readthedocs.org/en/latest/messaging.html>
- [22] N. D. Matsakis and F. S. Klock, “The Rust language,” *ACM SIGAda Ada Letters*, vol. 34, no. 3, pp. 103–104, 2014, doi: [10.1145/2692956.2663188](https://doi.org/10.1145/2692956.2663188).
- [23] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2nd ed. San Francisco: No Starch Press, 2023.
- [24] Tokio Contributors, “Tokio: An asynchronous runtime for Rust.” Accessed: September 1, 2026. [Online]. Доступно на <https://tokio.rs/>
- [25] I. Fette and A. Melnikov, “The WebSocket Protocol,” technical report RFC 6455, 2011. doi: [10.17487/RFC6455](https://doi.org/10.17487/RFC6455).
- [26] G. Bierman, M. Abadi, and M. Torgersen, “Understanding TypeScript,” in *ECOOP 2014 – Object-Oriented Programming*, Springer, 2014, pp. 257–281. doi: [10.1007/978-3-662-44202-9_11](https://doi.org/10.1007/978-3-662-44202-9_11).
- [27] Meta Platforms, Inc. and contributors, “React: The library for web and native user interfaces.” Accessed: September 1, 2026. [Online]. Доступно на <https://react.dev/>
- [28] Microsoft Corporation, “Monaco Editor: The code editor that powers VS Code.” Accessed: September 1, 2026. [Online]. Доступно на <https://microsoft.github.io/monaco-editor/>
- [29] Poimandres, “Zustand: A small, fast and scalable state-management solution.” Accessed: September 1, 2026. [Online]. Доступно на <https://zustand.docs.pmnd.rs/>
- [30] M. Weststrate, “Immer: Create the next immutable state by mutating the current one.” Accessed: September 1, 2026. [Online]. Доступно на <https://immerjs.github.io/immer/>
- [31] A. Haas *et al.*, “Bringing the web up to speed with WebAssembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2017, pp. 185–200. doi: [10.1145/3062341.3062363](https://doi.org/10.1145/3062341.3062363).
- [32] A. Y. Wang, Z. Wu, C. Brooks, and S. Oney, ““Don’t Step on My Toes”: Resolving Editing Conflicts in Real-Time Collaboration in Computational Notebooks,” in *2024 First IDE Workshop (IDE ’24)*, 2024. doi: [10.1145/3643796.3648453](https://doi.org/10.1145/3643796.3648453).
- [33] Jupyter Development Team, “Real-time collaboration without impersonation.” Accessed: August 25, 2026. [Online]. Доступно на <https://jupyterhub.readthedocs.io/en/stable/tutorial/collaboration-users.html>
- [34] Jupyter Development Team, “jupyter-collaboration: A Jupyter Server Extension Providing Support for Y Documents.” Accessed: August 25, 2026. [Online]. Доступно на <https://github.com/jupyterlab/jupyter-collaboration>

-
- [35] Jupyter Development Team, “jupyter_ydoc: Jupyter document structures for collaborative editing using Yjs/pycrdt.” Accessed: August 25, 2026. [Online]. Доступно на <https://jupyter-ydoc.readthedocs.io/en/latest/schema.html>
- [36] Jupyter Development Team, “Code Architecture — jupyter_collaboration documentation.” Accessed: August 25, 2026. [Online]. Доступно на <https://jupyterlab-realtime-collaboration.readthedocs.io/en/latest/developer/architecture.html>
- [37] y-crdt contributors, “pycrdt: Python bindings for Yrs.” Accessed: August 27, 2026. [Online]. Доступно на <https://pypi.org/project/pycrdt/>
- [38] W. Stein, “Collaborative Editing in CoCalc: OT, CRDT, or something else?.” Accessed: August 25, 2026. [Online]. Доступно на <https://blog.cocalc.com/2018/10/11/collaborative-editing.html>
- [39] N. Fraser, “Differential Synchronization.” Accessed: August 25, 2026. [Online]. Доступно на <https://neil.fraser.name/writing/sync/>
- [40] D. Burke and A. Kern, “Canvas, meet code: Building Figma's code layers.” Accessed: August 25, 2026. [Online]. Доступно на <https://www.figma.com/blog/building-figmas-code-layers/>
- [41] J. Gentle and M. Kleppmann, “Collaborative Text Editing with Eg-walker: Better, Faster, Smaller,” in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025. doi: 10.1145/3689031.3696076.
- [42] J. Wejdenstål, “dashmap: Blazing fast concurrent HashMap for Rust.” Accessed: August 30, 2026. [Online]. Доступно на <https://github.com/xacrimon/dashmap>
- [43] P. Butler, “fractional_index: An implementation of fractional indexing.” Accessed: August 30, 2026. [Online]. Доступно на https://github.com/jamsocket/fractional_index
- [44] spebern and contributors, “operational-transform: A library for Operational Transformation.” Accessed: August 30, 2026. [Online]. Доступно на <https://github.com/spebern/operational-transform-rs>
- [45] ZeroMQ Project, “zmqr.rs: A native Rust implementation of ZeroMQ.” Accessed: August 30, 2026. [Online]. Доступно на <https://github.com/zeromq/zmq.rs>

TODOs

1. [Додати скраћенице](#)
2. [Додати коришћене појмове](#)
3. [Додати скраћенице](#)
4. [Додати коришћене појмове](#)